

Computer Vision

Note complete del corso

Capitoli 1 – 19

Prof. Irene Amerini

Anno Accademico 2024/2025

Contents

1	Introduzione alla Computer Vision	5
1.1	Definizione e obiettivi	5
1.2	Rappresentazione di un'immagine	5
1.3	Computer Vision, Image Processing e Computer Graphics	5
1.4	Percezione visiva	6
1.5	Sfide principali	6
1.6	Breve contesto storico	6
1.7	Sfide aperte	6
2	Image Processing	7
2.1	Pipeline tipica di acquisizione a colori	7
2.2	Operatori puntuali (Point Processing)	7
2.2.1	Correzione gamma	8
2.2.2	Equalizzazione dell'istogramma	8
2.3	Filtraggio lineare	8
2.3.1	Cross-correlazione e convoluzione	9
2.3.2	Padding e gestione dei bordi	9
2.3.3	Esempi di filtri lineari	10
2.4	Filtraggio non lineare	10
2.4.1	Filtro bilaterale	11
2.4.2	Filtro mediana	11
2.4.3	Sogliatura (Thresholding)	11
2.5	Scalatura dell'immagine	11
2.5.1	Sottocampionamento e aliasing	11
2.5.2	Upsampling e interpolazione	12
2.6	Derivate dell'immagine e rilevamento dei bordi	12
2.6.1	Origine fisica dei bordi	12
2.6.2	Il gradiente dell'immagine	13
2.6.3	Derivate del secondo ordine: il Laplaciano	13
2.6.4	L'operatore di Sobel	14
2.6.5	La pipeline di Canny	14
3	Piramidi di immagini	16
3.1	Piramidi Gaussiane	16
3.2	Piramidi Laplaciane	17
3.2.1	Sommario delle formule	17
3.2.2	Ricostruzione	17
3.3	Applicazioni delle piramidi	18

3.3.1	Blending di immagini	18
3.3.2	Altre rappresentazioni multi-risoluzione	18
3.3.3	Rilevamento multi-scala	18
4	Dominio delle frequenze	19
4.1	Dominio delle frequenze e serie di Fourier	19
4.1.1	Costruire segnali da sinusoidi	19
4.2	La trasformata di Fourier	19
4.2.1	Definizione matematica	20
4.2.2	La trasformata discreta di Fourier (DFT)	20
4.3	Analisi di Fourier in 2D	20
4.4	Coppie della trasformata di Fourier e proprietà	21
4.5	Filtraggio nel dominio delle frequenze	21
4.5.1	Il teorema della convoluzione	21
4.6	Campionamento	22
4.7	Immagini ibride	22
5	Feature locali	24
5.1	Introduzione alle feature locali	24
5.2	Invarianza ed equivarianza	24
5.3	Applicazioni delle feature locali	25
5.4	Detection, description e matching	25
5.5	Rilevatore di angoli di Harris	25
5.5.1	La matematica del rilevamento di angoli	25
5.5.2	La matrice dei secondi momenti	25
5.5.3	Interpretazione della matrice dei secondi momenti	26
5.5.4	L'operatore di Harris e l'algoritmo	26
5.5.5	Proprietà e limitazioni	27
5.6	Rilevamento di blob invariante alla scala	27
6	Descrittori locali	28
6.1	Progettare descrittori di feature	28
6.1.1	Evoluzione di un descrittore	28
6.1.2	Confronto di istogrammi	28
6.2	Descrittore HOG (Histogram of Oriented Gradients)	29
7	Scale-Invariant Feature Transform (SIFT)	31
7.1	Rilevatore SIFT	31
7.1.1	Costruzione dello scale-space	31
7.1.2	Rilevamento degli estremi nello scale-space	31
7.1.3	Assegnazione dell'orientazione	32
7.2	Descrittore SIFT	32
7.3	Matching di feature	33
7.3.1	Ratio test di Lowe	33
7.4	Valutazione del matching	33
7.5	SURF: Speeded-Up Robust Features	33
7.6	Descrittori binari di feature	34
7.7	Deep local features	34

8	Trasformazioni e allineamento	35
8.1	Image warping	35
8.2	Tipi di trasformazioni 2D	35
8.2.1	Trasformazioni lineari	35
8.2.2	Trasformazioni affini	36
8.2.3	Trasformazioni proiettive (omografie)	36
8.3	Calcolo delle trasformazioni	37
8.3.1	Sistema lineare e minimi quadrati	37
8.3.2	Calcolo di un'omografia	37
8.4	Stima robusta con RANSAC	37
8.5	Implementazione del warping e blending	38
8.5.1	Forward vs. inverse warping	38
8.5.2	Blending	38
8.6	Punti di fuga	38
9	Riconoscimento	39
9.1	Compiti di riconoscimento	39
9.2	Classificazione di immagini	39
9.2.1	Bag-of-Words (BoW)	39
9.3	Reti Neurali Convoluzionali (CNN)	40
9.3.1	Output e funzione di loss	40
9.3.2	Architetture principali	40
9.4	Segmentazione semantica	40
9.5	Rilevamento di oggetti	41
9.5.1	Rilevatori a due stadi	41
9.5.2	Rilevatori a stadio singolo	41
9.5.3	Feature Pyramid Networks (FPN)	41
9.5.4	Sliding window e HOG	41
9.6	Estensioni	41
10	Flusso ottico	43
10.1	Introduzione	43
10.2	Il problema dell'apertura	43
10.3	Stima del flusso ottico	44
10.3.1	Equazione di costanza della luminosità	44
10.4	Lucas-Kanade	44
10.4.1	Gestione dei grandi moti: coarse-to-fine	44
10.5	Horn-Schunck	45
10.6	Deep learning per il flusso ottico	45
11	Calibrazione della camera	46
11.1	Il modello della camera	46
11.1.1	Pinhole e lente sottile	46
11.1.2	Proiezione prospettica	46
11.1.3	Dal piano immagine ai pixel	46
11.1.4	La matrice della camera	47
11.2	Calibrazione della camera	47
11.2.1	Estrazione dei parametri	47
11.3	Punti di fuga	48

11.4	Visione stereo	48
11.4.1	Profondità dalla disparità	48
11.4.2	Matching stereo	48
11.4.3	Stereo come minimizzazione di energia	48
11.5	Stereo attivo e luce strutturata	48
12	Geometria epipolare	49
12.1	Introduzione	49
12.2	La matrice essenziale	50
12.2.1	Derivazione	50
12.3	La matrice fondamentale	50
12.4	Stima della matrice fondamentale	51
12.4.1	Algoritmo a 8 punti	51
12.4.2	Algoritmo a 8 punti normalizzato	51
12.5	Triangolazione	51
12.6	Rettificazione stereo e DSI	52
12.7	Da $\mathbf{F}$ a moto della camera	52
13	Vision Transformers e Architetture Avanzate	53
13.1	Dal NLP alla Visione: Motivazione	53
13.2	Meccanismo di Self-Attention	53
13.2.1	Multi-Head Attention (MHA)	54
13.2.2	Positional Encoding	54
13.2.3	Struttura del Transformer Encoder Block	54
13.3	Vision Transformer (ViT)	54
13.4	Swin Transformer: Shifted-Window Attention	55
13.5	DETR: End-to-End Object Detection with Transformers	55
14	Modelli Generativi Profondi: GANs e Diffusione	57
14.1	Formulazione generale	57
14.2	Generative Adversarial Networks (GANs)	57
14.3	Denoising Diffusion Probabilistic Models (DDPM)	58
14.3.1	Forward Process (Noising)	58
14.3.2	Reverse Process (Denoising)	58
15	Deep Learning per Video e Dati Temporali	59
15.1	Il Video come Segnale Spazio-Temporale	59
15.2	Architetture per Video Recognition	59
15.3	TimeSformer: Video Vision Transformer	59
16	Vision-Language Models (VLMs)	61
16.1	Motivazione e Strategie di Fusione	61
16.2	CLIP: Contrastive Language-Image Pre-training	61
16.2.1	Zero-Shot Transfer	62
17	Medical Imaging e Segmentazione Avanzata	63
17.1	Il Medical Data Gap	63
17.2	Architetture di Segmentazione Biomedica	63
17.3	Funzioni di Loss per Dati Sbilanciati	63

17.4	Metriche di Valutazione	64
18	Monocular Depth Estimation (MDE)	65
18.1	Definizione e Natura Ill-Posed	65
18.2	Indizi Monoculari (Monocular Cues)	65
18.3	Depth Anything v2	65
19	Efficienza nel Deep Learning	66
19.1	Training Efficiente vs. Inferenza Efficiente	66
19.2	Tecniche di Inferenza Efficiente	66
19.3	Metriche e Trade-off	66

Chapter 1

Introduzione alla Computer Vision

1.1 Definizione e obiettivi

La **Computer Vision** mira a convertire luce in significato, estraendo informazioni geometriche, semantiche e temporali da immagini e video. A differenza dell'*Image Processing*, che trasforma immagini in altre immagini, la Computer Vision cerca di inferire proprietà del mondo a partire da dati visivi.

Problema inverso mal posto

La Computer Vision può essere vista come un **problema inverso mal posto**: date una o più immagini 2D, l'obiettivo è recuperare informazioni sulla scena 3D sottostante. Molte scene 3D diverse, condizioni di illuminazione e configurazioni di oggetti possono produrre immagini 2D simili; gli algoritmi necessitano quindi di assunzioni, vincoli e prior appresi per interpretare correttamente le immagini.

1.2 Rappresentazione di un'immagine

Un'immagine può essere interpretata in due modi:

- **Dominio discreto**: griglia di valori di intensità in $[0, 2^{\text{bit}}]$.
- **Dominio continuo**: una funzione $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, dove $f(x, y)$ fornisce l'intensità nella posizione spaziale (x, y) . Un'immagine digitale è semplicemente una versione discreta, campionata e quantizzata di questa funzione continua.

1.3 Computer Vision, Image Processing e Computer Graphics

- **Image Processing**: trasforma immagini in altre immagini (filtraggio, riduzione del rumore, rilevamento di bordi, ...).
- **Computer Vision**: inferisce informazioni dal mondo a partire da immagini (identità degli oggetti, struttura 3D, moto, posa, semantica della scena).
- **Computer Graphics**: risolve il *problema diretto* — data una scena 3D con

oggetti, materiali, illuminazione e parametri della telecamera, genera un'immagine 2D. La Computer Vision risolve il *problema inverso*.

1.4 Percezione visiva

La percezione visiva *non* è una semplice misurazione delle intensità dei pixel. La stessa intensità misurata può essere percepita in modo diverso a seconda del contesto, delle ombre, della geometria e delle conoscenze pregresse. Ciò spiega perché i sistemi di visione hanno bisogno di modelli, assunzioni e vincoli per interpretare correttamente le immagini.

1.5 Sfide principali

Variazione di punto di vista, variazione di scala, deformazione, occlusione, variazioni di illuminazione, cluttering di sfondo, variazione intra-classe, moto e ambiguità locale.

1.6 Breve contesto storico

Milestones importanti: stereo e fotogrammetria precoci; shape from shading; flusso ottico; matrice essenziale; Markov Random Fields; feature locali (SIFT); reti neurali convoluzionali; ImageNet e AlexNet; metodi di rilevamento di oggetti (YOLO, Mask R-CNN); modelli di diffusione e modelli multimodali.

1.7 Sfide aperte

Apprendimento non supervisionato e auto-supervisionato, apprendimento interattivo, robustezza, generalizzazione, bias induttivi, interpretabilità, costi computazionali e problemi etici/legali.

Chapter 2

Image Processing

2.1 Pipeline tipica di acquisizione a colori

Un'immagine digitale è il risultato di una pipeline di acquisizione ed elaborazione. La luce viene catturata da un sensore CCD/CMOS dotato di un **color filter array** (solitamente un pattern di Bayer). Le misurazioni grezze del sensore vengono poi elaborate attraverso: guadagno ISO, elaborazione raw, demosaicing, bilanciamento del bianco, trasformazione dello spazio colore, riduzione del rumore, mappatura a sRGB e infine compressione (es. JPEG).

Importanza per la Computer Vision

Le immagini usate dagli algoritmi di CV spesso non sono misurazioni fisiche grezze: sono già state tone-mappate, corrette per il colore, derumorose e compresse. Molti algoritmi assumono che le intensità siano in uno *spazio lineare*.

2.2 Operatori puntuali (Point Processing)

Operatori puntuali

Gli operatori puntuali modificano il valore di ciascun pixel **indipendentemente dai suoi vicini**. Il nuovo valore di un pixel in posizione (x, y) dipende solo dal valore originale in quella posizione.

Esempi:

- **Luminosità:** $g(x, y) = f(x, y) + b$. Aggiungere una costante b schiarisce o scurisce l'immagine.
- **Contrasto:** $g(x, y) = a \cdot f(x, y)$. Moltiplicare per $a > 1$ aumenta il contrasto; $a < 1$ lo riduce.
- **Inversione:** $g(x, y) = 255 - f(x, y)$ per un'immagine a 8 bit.
- **Flip orizzontale:** $g(x, y) = f(-x, y)$.

2.2.1 Correzione gamma

La correzione gamma **aggiusta la luminosità** di un'immagine visualizzata su schermo per renderla più naturale all'occhio:

$$V_{\text{out}} = A \cdot V_{\text{in}}^\gamma$$

dove V_{in} è la tensione d'ingresso, V_{out} è l'uscita regolata, A è una costante e tipicamente $\gamma = 2.2$.

Gamma e spazio sRGB

La maggior parte delle immagini è memorizzata in spazio sRGB, già tone-mappato e *non* lineare rispetto alla luminanza della scena. Molti algoritmi di CV assumono uno spazio lineare. In assenza della curva di risposta della camera, si usa tipicamente $\gamma = 2.2$ come approssimazione (non fisicamente esatta, ma spesso sufficiente).

2.2.2 Equalizzazione dell'istogramma

L'**Histogram Equalization (HE)** aggiusta il contrasto globale redistribuendo i valori di intensità per produrre un istogramma più piatto, rendendo i dettagli più visibili.

1. **Calcolo dell'istogramma:** conta il numero di pixel per ogni livello di intensità.

$$p(r_k) = \frac{\text{\#pixel con intensità } r_k}{\text{\#pixel totali}}$$

2. **Calcolo della CDF:** mappa ogni intensità di input a un valore in $[0, 1]$.

$$\text{CDF}(r_k) = \sum_{i=0}^k p(r_i)$$

3. **Scalatura:** la CDF viene scalata all'intero range di intensità e usata come lookup table.

$$r'_k = \text{round}\left(\frac{(\text{CDF}(r_k) - \text{CDF}_{\min})(L - 1)}{N - \text{CDF}_{\min}}\right)$$

dove L è il massimo valore di pixel, N il numero totale di pixel e $\text{CDF}_{\min}$ il minimo valore non-zero della CDF.

Local Histogram Equalization (AHE): calcola una trasformazione per ciascun pixel basandosi sull'istogramma di un piccolo intorno locale, migliorando i dettagli locali a scapito della complessità computazionale.

CLAHE — Contrast Limited AHE

Un limite dell'AHE è che può amplificare fortemente il rumore in regioni quasi uniformi. La **CLAHE** risolve il problema agganciando i picchi dell'istogramma tramite un *clip limit* predefinito e redistribuendo i pixel tagliati. L'interpolazione bilineare tra tile vicine evita artefatti a blocchi.

2.3 Filtraggio lineare

Filtraggio lineare

Il filtraggio crea una nuova immagine in cui il valore di ciascun pixel è una **funzione dei suoi vicini** nell'immagine originale. Più specificamente, il *filtraggio lineare* calcola ciascun nuovo pixel come **combinazione lineare** (somma pesata) dei suoi vicini; l'insieme dei pesi è detto **kernel** o **filtro**.

2.3.1 Cross-correlazione e convoluzione

Queste sono le due operazioni primarie di filtraggio lineare.

Cross-correlazione: l'output $G[i, j]$ è la somma dei prodotti element-wise tra il kernel H e il vicinato dell'immagine F centrato in (i, j) :

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

Convoluzione: uguale alla cross-correlazione, tranne che il kernel H viene ribaltato orizzontalmente e verticalmente prima di essere applicato:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

Proprietà della convoluzione

- **Commutativa:** $a * b = b * a$
- **Associativa:** $(a * b_1) * b_2 = a * (b_1 * b_2)$. Applicare due filtri in sequenza equivale a convolverli prima in un unico filtro.
- **Distributiva sull'addizione.**

In pratica, se il kernel è simmetrico, convoluzione e cross-correlazione sono identiche.

2.3.2 Padding e gestione dei bordi

Quando il kernel è centrato su un pixel vicino al bordo dell'immagine, la finestra del kernel può sconfinare oltre il bordo. Strategie di padding:

- **Zero:** si aggiungono zeri all'esterno.
- **Costante:** si aggiunge un valore costante c .
- **Clamp / replica:** si replica il bordo.
- **Reflection / mirror:** si riflette verso l'interno.
- **Symmetric:** si riflette includendo il bordo.

Dimensione dell'output. Per una convoluzione con input di dimensione N , kernel di dimensione K , padding P e stride S :

$$\dim_{\text{out}} = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1$$

Casi speciali (stride $S = 1$):

- **full**: output più grande dell'input.
- **same**: output con stessa dimensione dell'input ($P = \lfloor K/2 \rfloor$).
- **valid**: nessun padding, output più piccolo ($\dim_{\text{out}} = N - K + 1$).

2.3.3 Esempi di filtri lineari

Filtro box (mean filter). Sostituisce ogni pixel con la media dei valori nel suo vicinato rettangolare. Il kernel è composto da tutti 1, normalizzati per il numero di elementi (es. diviso 9 per un kernel 3×3). Produce sfocatura (blurring).

Filtro Gaussiano. Un filtro passa-basso più sofisticato che usa un kernel con pesi estratti da una distribuzione Gaussiana 2D. I pixel più vicini al centro ricevono pesi maggiori:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Il parametro σ controlla la quantità di sfocatura.

Proprietà del filtro Gaussiano

- Convolvere un Gaussiano con un altro Gaussiano produce un nuovo Gaussiano più largo.
- Un kernel Gaussiano 2D è **separabile**: la convoluzione 2D può essere eseguita come due convoluzioni 1D (una orizzontale, una verticale), molto più efficienti computazionalmente.

Costo computazionale dei filtri separabili

Un filtro 2D è separabile quando la sua matrice kernel ha rango 1. Per un'immagine $M \times N$ e un kernel $L \times L$:

- Convoluzione 2D standard: $\mathcal{O}(L^2MN)$
- Implementazione separabile (due conv. 1D): $\mathcal{O}(2LMN)$

Filtro di nitidezza (unsharp masking). Un effetto di nitidezza si ottiene amplificando i dettagli dell'immagine. I dettagli vengono estratti sottraendo la versione sfocata all'originale, poi riaggiunti:

$$F_{\text{sharp}} = F_{\text{orig}} + \alpha(F_{\text{orig}} - F_{\text{blur}})$$

dove α è la forza di nitidezza ($F_{\text{sharp}} = F_{\text{orig}}$ per $\alpha = 0$). Si può implementare con un unico filtro che approssima un'impulso scalato meno un Gaussiano.

2.4 Filtraggio non lineare

I filtri non lineari eseguono operazioni di vicinato, ma il nuovo valore del pixel *non* è una combinazione lineare dei suoi vicini.

2.4.1 Filtro bilaterale

È un filtro di smoothing avanzato che **preserva i bordi**. Un filtro Gaussiano standard media i pixel indipendentemente dal loro contenuto, causando sfocatura attraverso i bordi degli oggetti. Il filtro bilaterale risolve questo problema pesando i pixel vicini in base sia alla loro **distanza spaziale** che alla loro **differenza di intensità**:

$$h[m, n] = \frac{1}{W_{mn}} \sum_{k,l} \underbrace{g[k, l]}_{\text{kernel spaziale}} \underbrace{r_{mn}[k, l]}_{\text{kernel di range}} f[m + k, n + l]$$

dove:

- $g[k, l]$: kernel Gaussiano spaziale, pesa i pixel in base alla distanza Euclidea dal pixel centrale (m, n) .
- $r_{mn}[k, l]$: kernel di range (o di intensità), pesa i pixel in base alla differenza di intensità dal pixel centrale $f[m, n]$.
- $W_{mn} = \sum_{k,l} g[k, l] r_{mn}[k, l]$: fattore di normalizzazione.

Un pixel contribuisce alla media solo se è sia spazialmente vicino sia fotometricamente simile. Il filtro bilaterale elimina il rumore nelle regioni piatte ma preserva i bordi netti. Applicazioni: denoising di alta qualità, ritocco fotografico, effetto cartoon.

2.4.2 Filtro mediana

Sostituisce ogni pixel con il valore **mediano** di tutti i pixel nel suo vicinato.

Mediana vs. media

Il filtro mediana rimuove il **rumore sale e pepe**: i valori anomali (*outlier*) vengono ignorati dalla mediana, mentre la media diffonde il rumore nei pixel circostanti e sfoca i bordi.

2.4.3 Sogliatura (Thresholding)

Operazione non lineare semplice che converte un'immagine in scala di grigi in un'immagine binaria:

$$g(m, n) = \begin{cases} 255 & \text{se } f(m, n) > A \\ 0 & \text{altrimenti} \end{cases}$$

2.5 Scalatura dell'immagine

2.5.1 Sottocampionamento e aliasing

Per ridurre le dimensioni di un'immagine, un approccio ingenuo è il **sottocampionamento**: eliminare righe e colonne.

Aliasing

Il semplice sottocampionamento può causare **aliasing**: un effetto in cui dettagli fini e pattern nell'immagine originale vengono distorti in nuovi pattern errati nell'immagine ridotta. Questo accade quando la **frequenza di campionamento**

è **troppo bassa** per catturare le frequenze più alte nell'immagine. Per evitarlo, la frequenza di campionamento deve essere almeno il doppio della frequenza massima presente nell'immagine (**frequenza di Nyquist**):

$$f_s \geq 2 f_{\max}$$

Anti-aliasing: il modo corretto per ridurre un'immagine è applicare prima un **filtro passa-basso** (es. blur Gaussiano) per rimuovere i dettagli ad alta frequenza che non possono essere rappresentati alla risoluzione inferiore. Dopo la sfocatura, l'immagine può essere sottocampionata in modo sicuro.

Aliasing nei sistemi pratici

L'aliasing appare anche nelle implementazioni pratiche del downsampling nei framework di deep learning. Le operazioni di pooling nelle CNN possono ridurre l'*equivarianza* rispetto alle traslazioni, rendendo le predizioni instabili a piccole traslazioni dell'immagine. Il *anti-aliased pooling* risolve il problema inserendo un filtro passa-basso prima del sottocampionamento.

2.5.2 Upsampling e interpolazione

Per aumentare le dimensioni di un'immagine è necessario **interpolare** nuovi pixel:

- **Nearest-neighbor:** replica i pixel; il più semplice ma produce risultati a blocchi.
- **Bilineare:** calcola ciascun nuovo pixel come media pesata dei 4 pixel più vicini nell'immagine originale; risultati più fluidi.
- **Bicubica:** usa un vicinato di 16 pixel per un risultato ancora più fluido.

Super-risoluzione moderna

Tecniche avanzate usano reti neurali per imparare come generare dettagli ad alta risoluzione, oppure metodi multi-immagine che catturano foto leggermente shiftate e le combinano per ricostruire un'immagine a risoluzione maggiore.

2.6 Derivate dell'immagine e rilevamento dei bordi

Bordi

I **bordi** (*edges*) sono posizioni in un'immagine con un **rapido cambiamento di intensità**. Sono caratteristiche fondamentali per comprendere il contenuto dell'immagine.

2.6.1 Origine fisica dei bordi

I bordi possono essere causati da diversi fenomeni fisici:

- Discontinuità della normale alla superficie.
- Discontinuità di profondità.

- Discontinuità di colore o materiale della superficie.
- Discontinuità di illuminazione (es. ombre).

2.6.2 Il gradiente dell'immagine

Poiché un'immagine è una funzione 2D $f(x, y)$, possiamo analizzare le variazioni di intensità calcolando le sue derivate, approssimate tramite convoluzione con piccoli kernel.

- La **derivata parziale** $\frac{\partial f}{\partial x}$ si approssima convolvendo con un filtro come $[-1, 1]$ oppure $[1, -1]$. (Analogamente per $\frac{\partial f}{\partial y}$ in verticale.)
- Il **gradiente** $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$ è un vettore che punta nella direzione del più rapido aumento di intensità.
- La **magnitudine del gradiente**:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

misura la forza del bordo.

- L'**orientazione del gradiente**:

$$\theta = \text{atan2}\left(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}\right)$$

fornisce la direzione del bordo.

Gestione del rumore. Applicare direttamente filtri derivativi è altamente sensibile al rumore. La soluzione è **prima uniformare l'immagine con un filtro Gaussiano**. Grazie alla proprietà associativa della convoluzione:

$$\frac{\partial}{\partial x}(G * f) = \left(\frac{\partial}{\partial x}G\right) * f$$

Si ottiene il filtro **Derivative of Gaussian (DoG)**: invece di sfocare l'immagine e poi prenderne la derivata, si prende la derivata del kernel Gaussiano e si convolve con essa l'immagine in un'unica operazione efficiente.

Nota terminologica: DoG

In rilevamento dei bordi, l'espressione *Derivative of Gaussian* non va abbreviata in DoG, perché in rilevamento di feature e SIFT, DoG significa tipicamente *Difference of Gaussians*. Si tratta di concetti distinti.

2.6.3 Derivate del secondo ordine: il Laplaciano

La derivata del secondo ordine può anch'essa essere usata per trovare i bordi. Il **Laplaciano** di un'immagine è:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

che è uno scalare.

- In un segnale 1D, un bordo corrisponde a un picco nella prima derivata e a un **zero-crossing** nella seconda derivata.
- Combinando il Laplaciano con lo smoothing Gaussiano si crea un filtro **Laplacian of Gaussian (LoG)**. I bordi vengono localizzati trovando gli zero-crossing nell'immagine filtrata.

Zero-crossing vs. DoG

Gli zero-crossing (LoG) sono più accurati nel localizzare i bordi, ma il DoG è computazionalmente più efficiente. La prima derivata in corrispondenza di un bordo corrisponde a un massimo, mentre la seconda derivata è zero.

2.6.4 L'operatore di Sobel

L'operatore di Sobel è un'approssimazione efficiente del DoG. È un **filtro separabile** composto da un kernel derivativo 1D e un kernel di smoothing 1D. Il filtro Sobel orizzontale combina $[1, 2, 1]$ (smoothing) e $[1, 0, -1]$ (derivativo).

Altri filtri derivativi comuni: **Prewitt, Roberts, Scharr**.

Normalizzazione di Sobel

I mask di Sobel standard sono spesso scritti senza il fattore di normalizzazione $1/8$. Questo non influisce sul rilevamento dei bordi se si usano solo risposte relative o soglie, ma il fattore è necessario per ottenere la scala corretta della magnitudine del gradiente.

2.6.5 La pipeline di Canny

Il rilevatore di Canny è un algoritmo multi-stadio considerato il metodo classico standard per il rilevamento dei bordi.

1. **Filtraggio**: si uniforma l'immagine con un filtro Gaussiano e si calcolano magnitudine e orientazione del gradiente usando un filtro DoG (o un'approssimazione come Sobel).
2. **Non-maximum suppression**: per ogni pixel, si verifica se la sua magnitudine è un massimo locale nella direzione del suo gradiente; se no, lo si sopprime impostandolo a 0. Ciò produce bordi sottili, larghi un pixel.
3. **Linking e sogliatura isteresi**: si usano due soglie (alta e bassa).
 - Pixel con magnitudine del gradiente sopra la soglia alta: bordi *forti*, conservati.
 - Pixel con magnitudine tra soglia bassa e alta: bordi *deboli*, conservati solo se connessi a un pixel di bordo forte.
 - Pixel sotto la soglia bassa: scartati.

Criteri di ottimalità del rilevatore di Canny

Un rilevatore di bordi ottimale deve soddisfare tre criteri:

1. **Buona rilevazione:** pochi falsi positivi e falsi negativi.
2. **Buona localizzazione:** i bordi rilevati devono essere vicini alla vera posizione del bordo.
3. **Risposta singola:** un bordo fisico deve generare un solo bordo rilevato.

Il comportamento dipende principalmente da tre parametri: la scala di smoothing Gaussiano σ , la soglia alta e la soglia bassa. Un σ più grande sopprime i dettagli fini e rileva bordi su scala maggiore; un σ più piccolo preserva bordi più fini ma è più sensibile al rumore.

Chapter 3

Piramidi di immagini

3.1 Piramidi Gaussiane

Piramide Gaussiana

Una **piramide Gaussiana** è una rappresentazione multi-risoluzione dell'immagine in cui ogni livello è creato applicando un filtro Gaussiano e sottocampionando il livello precedente di un fattore 2, producendo immagini progressivamente più piccole e sfocate, utili per la **ricerca coarse-to-fine**.

Man mano che si sale di livello nella piramide (risoluzioni minori), i dettagli ad alta frequenza vengono eliminati, mentre le componenti a bassa frequenza e le regioni uniformi vengono preservate.

Perché si usa il filtro Gaussiano

Il Gaussiano è usato perché è **auto-riproduttore**: convolvere un Gaussiano con un altro Gaussiano produce un nuovo Gaussiano con varianza maggiore. Questo rende possibile lo smoothing incrementale tra i livelli della piramide. Una volta filtrata passa-basso l'immagine, sono necessari meno pixel per rappresentare il contenuto rimanente senza aliasing severo.

La sfocatura è con perdita di informazione

Non è possibile ricostruire l'immagine originale ad alta risoluzione a partire da un livello a risoluzione inferiore.

Dimensione di una piramide Gaussiana

Se ogni livello è sottocampionato di un fattore 2, sia la larghezza che l'altezza vengono dimezzate $\Rightarrow$ il livello successivo ha $1/4$ dei pixel del precedente. Il numero totale di pixel dell'intera piramide:

$$N_{\text{tot}} = N + \frac{N}{4} + \frac{N}{16} + \dots = N \sum_{i=0}^{\infty} \left(\frac{1}{4}\right)^i = N \cdot \frac{1}{1 - 1/4} = \frac{4}{3}N$$

L'intera piramide richiede solo circa il $\frac{4}{3}$ dello storage dell'immagine originale.

3.2 Piramidi Laplaciane

Piramide Laplaciana

La **piramide Laplaciana** è una **rappresentazione senza perdita** che memorizza i dettagli ad alta frequenza persi tra due livelli di una piramide Gaussiana, calcolando un'immagine *residuo*.

Si definiscono due operazioni fondamentali:

- **REDUCE**(I): crea il livello successivo G_{i+1} della piramide Gaussiana sfocando e poi sottocampionando G_i :

$$G_{i+1} = \text{REDUCE}(G_i) = \text{downsample}(\text{blur}(G_i))$$

- **EXPAND**(I): predice un'immagine a risoluzione maggiore G_i da G_{i+1} a risoluzione minore, tramite upsampling e sfocatura:

$$\text{EXPAND}(G_{i+1}) = \text{blur}(\text{upsample}(G_{i+1}))$$

Il **residuo** L_i forma un livello della piramide Laplaciana:

$$L_i = G_i - \text{EXPAND}(G_{i+1})$$

Ogni residuo contiene principalmente i dettagli persi nel passaggio da un livello Gaussiano a quello successivo (bordi, contorni, texture fine).

La rappresentazione completa della piramide Laplaciana è $\{L_0, L_1, \dots, L_{n-1}\}$ più il livello Gaussiano più piccolo G_n .

Perché “Laplaciana”? La creazione di ogni residuo (immagine meno versione sfocata, differenza di Gaussiani) approssima l'operatore LoG:

$$\underbrace{G_i - \text{EXPAND}(G_{i+1})}_{\text{Difference of Gaussians}} \approx \underbrace{\nabla^2 G}_{\text{Laplacian of Gaussian}}$$

3.2.1 Sommario delle formule

$$G_{i+1} = \text{REDUCE}(G_i) \quad L_i = G_i - \text{EXPAND}(G_{i+1}) \quad G_i = L_i + \text{EXPAND}(G_{i+1})$$

3.2.2 Ricostruzione

L'immagine originale può essere **ricostruita perfettamente** dalla piramide Laplaciana invertendo il processo:

1. Si parte dall'immagine al livello più alto, G_n .
2. Per ogni livello i da $n - 1$ fino a 0:

$$G_i = \text{EXPAND}(G_{i+1}) + L_i$$

3. Il risultato finale G_0 è l'immagine originale perfettamente ricostruita.

In assenza di errori di quantizzazione e approssimazione numerica, la piramide Laplaciana consente **ricostruzione perfetta**. Per memorizzare questa ricostruzione si devono conservare: le immagini residuo a ogni livello; l'immagine più piccola al vertice della piramide Gaussiana.

3.3 Applicazioni delle piramidi

Le piramidi di immagini sono uno strumento fondamentale usato in: compressione di immagini, denoising, registrazione multi-scala (allineamento immagini da grossolano a fine) e rilevamento multi-scala (trovare oggetti a varie scale).

3.3.1 Blending di immagini

Le piramidi Laplaciane forniscono una tecnica potente per fondere due immagini in modo fluido, evitando la giunzione netta che risulterebbe da un semplice taglia-e-incolla.

Algoritmo di blending piramidale

Per fondere due immagini A e B usando un'immagine maschera M :

1. Costruire le piramidi Laplaciane per le due immagini sorgente, LA e LB .
2. Costruire una piramide Gaussiana per la maschera, GM . I valori della maschera (da 0 a 1) fungono da pesi di blending a ogni scala.
3. Per ogni livello j , creare un livello Laplaciano combinato LC_j come media pesata:

$$LC_j(x, y) = GM_j(x, y) \cdot LA_j(x, y) + (1 - GM_j(x, y)) \cdot LB_j(x, y)$$

4. Collassare la piramide Laplaciana combinata LC per ottenere l'immagine finale.

3.3.2 Altre rappresentazioni multi-risoluzione

Altre rappresentazioni multi-risoluzione includono le **piramidi orientabili** (*steerable pyramids*) e le **wavelet**.

- Le piramidi orientabili memorizzano risposte orientate multiple a ogni scala.
- Le wavelet decompongono l'immagine in approssimazioni a bassa frequenza e dettagli ad alta frequenza, spesso separati in componenti orizzontali, verticali e diagonali.

3.3.3 Rilevamento multi-scala

Nel rilevamento multi-scala, lo stesso template o rilevatore a dimensione fissa viene applicato a livelli diversi della piramide. Poiché gli oggetti appaiono a dimensioni diverse a seconda del livello, un rilevatore che funziona a una dimensione fissa di template può rilevare oggetti di diverse dimensioni apparenti scansionando la piramide.

Chapter 4

Dominio delle frequenze

4.1 Dominio delle frequenze e serie di Fourier

Qualsiasi funzione univariata può essere riscritta come somma pesata di seni e coseni di frequenze diverse, sotto certe condizioni.

4.1.1 Costruire segnali da sinusoidi

Il mattone di base per l'analisi di Fourier è la sinusoide:

$$A \sin(\omega x + \phi)$$

definita da tre parametri chiave:

- **Ampiezza** (A): la forza o intensità dell'onda.
- **Frequenza angolare** (ω): determina la velocità di oscillazione. Il periodo è $T = 2\pi/|\omega|$.
- **Fase** (ϕ): la posizione iniziale o offset della sinusoide.

Il **teorema di Fourier** afferma che sommando abbastanza sinusoidi con frequenze e ampiezze diverse, si può approssimare qualsiasi segnale periodico.

Lo **spettro di frequenza** è un grafico che mostra l'ampiezza di ciascuna componente di frequenza presente nel segnale. Il valore alla frequenza zero è detto **componente DC** e rappresenta il valore medio del segnale.

4.2 La trasformata di Fourier

Le serie di Fourier rappresentano funzioni periodiche. La *trasformata di Fourier* estende questa idea alle **funzioni non periodiche**:

Trasformata di Fourier

La trasformata di Fourier converte un segnale $f(x)$ dal dominio spaziale a una funzione $F(\omega)$ nel dominio delle frequenze. $F(\omega)$ è a valori complessi:

$$F(\omega) = R(\omega) + i I(\omega)$$

L'ampiezza e la fase si ricavano dalla parte reale e immaginaria:

$$A(\omega) = \sqrt{R(\omega)^2 + I(\omega)^2}, \quad \phi(\omega) = \text{atan2}(I(\omega), R(\omega))$$

L'uso di `atan2` è preferibile all'arcotangente semplice perché gestisce correttamente il quadrante del numero complesso. La trasformazione è invertibile.

4.2.1 Definizione matematica

Trasformata diretta (Spaziale $\rightarrow$ Frequenza):

$$F(\omega) = \int_{-\infty}^{+\infty} f(x) e^{-i\omega x} dx$$

Trasformata inversa (Frequenza $\rightarrow$ Spaziale):

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} F(\omega) e^{i\omega x} d\omega$$

La connessione con le sinusoidi è data dalla **formula di Eulero**: $e^{i\theta} = \cos \theta + i \sin \theta$. La trasformata misura essenzialmente quanta di una data frequenza seno e coseno è presente nel segnale $f(x)$.

4.2.2 La trasformata discreta di Fourier (DFT)

Per segnali e immagini digitali si usa la *Discrete Fourier Transform (DFT)*. Per un segnale 1D $f(x)$ di lunghezza N :

$$F(k) = \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

Può essere espressa come moltiplicazione matriciale $\mathbf{F} = \mathbf{W}\mathbf{f}$.

Fast Fourier Transform (FFT)

Il calcolo diretto della DFT tramite moltiplicazione matriciale ha complessità $\mathcal{O}(N^2)$. La **Fast Fourier Transform (FFT)** calcola la DFT in $\mathcal{O}(N \log N)$, rendendola pratica per l'elaborazione delle immagini.

4.3 Analisi di Fourier in 2D

La trasformata di Fourier si estende a 2D per analizzare le immagini. Un'immagine $f(x, y)$ viene trasformata nella sua rappresentazione in frequenza $F(u, v)$, dove u e v sono le frequenze orizzontale e verticale:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Nello spettro 2D di Fourier (visualizzato con il DC al centro):

- Le **basse frequenze** sono vicino al centro e corrispondono a strutture lisce su larga scala.
- Le **alte frequenze** sono lontane dal centro e corrispondono a dettagli fini, bordi e texture.
- L'orientazione delle linee nello spettro corrisponde all'orientazione dei bordi nell'immagine (es. i bordi verticali creano linee orizzontali nel dominio delle frequenze).

L'importanza della fase

- L'**ampiezza** della DFT rappresenta quanta di una particolare frequenza è presente nell'immagine $\Rightarrow$ manca di informazione spaziale.
- La **fase** della DFT rappresenta la disposizione spaziale di ogni componente di frequenza $\Rightarrow$ definisce la posizione di bordi, linee, angoli e texture.
- Se si ricostruisce un'immagine usando solo la fase, il risultato è riconoscibile ma distorto, con strane variazioni di intensità. La fase è generalmente più importante dell'ampiezza per la ricostruzione visiva.

4.4 Coppie della trasformata di Fourier e proprietà

Coppie importanti:

- Funzione *box* nel dominio spaziale $\leftrightarrow$ funzione *sinc* nel dominio delle frequenze.
- Gaussiano nel dominio spaziale $\leftrightarrow$ Gaussiano nel dominio delle frequenze.
- Per dualità: *sinc* nel dominio spaziale $\leftrightarrow$ *box* nel dominio delle frequenze.

Proprietà di scaling: espandere un segnale nel dominio spaziale comprime la sua trasformata nel dominio delle frequenze, e viceversa. Strutture lisce su larga scala $\leftrightarrow$ basse frequenze; dettagli piccoli e variazioni rapide $\leftrightarrow$ alte frequenze.

4.5 Filtraggio nel dominio delle frequenze

4.5.1 Il teorema della convoluzione

Teorema della convoluzione

La convoluzione nel dominio spaziale è equivalente alla moltiplicazione element-wise nel dominio delle frequenze:

$$\mathcal{F}\{g * h\} = \mathcal{F}\{g\} \cdot \mathcal{F}\{h\}$$

Equivalentemente, la moltiplicazione nel dominio delle frequenze corrisponde alla convoluzione nel dominio spaziale.

Invece di eseguire una costosa convoluzione nel dominio spaziale, si può:

1. Trasformare immagine e kernel nel dominio delle frequenze tramite FFT.
2. Moltiplicarli element-wise.

3. Ritrasformare il risultato nel dominio spaziale tramite FFT inversa.

Esempi di filtraggio:

- **Filtraggio passa-basso:** moltiplicare lo spettro per una maschera che preserva le basse frequenze (vicino al centro) e attenua le alte frequenze. Un Gaussiano è un eccellente filtro passa-basso.
- **Filtraggio passa-alto:** attenuare le basse frequenze e preservare le alte (per nitidezza o rilevamento bordi).
- **Filtraggio passa-banda:** preservare un anello di frequenze per isolare feature a una scala specifica.

Filtro box vs. filtro Gaussiano

Il **filtro box** è brusco nel dominio spaziale. La sua trasformata di Fourier è una **funzione sinc** con sidelobes significativi. Queste caratteristiche non ideali producono un taglio di frequenza imperfetto e possono causare **artefatti di ringing** nell'immagine sfocata risultante.

Il **filtro Gaussiano** è regolare sia nel dominio spaziale che in quello delle frequenze, garantendo un'attenuazione pulita. Un filtro passa-basso ideale (taglio netto in frequenza) produce anch'esso ringing perché la sua risposta impulsiva nel dominio spaziale è una sinc.

In pratica, il supporto spaziale di un mask Gaussiano viene scelto pari a circa **tre volte la deviazione standard**, in modo da includere la maggior parte della campana Gaussiana.

4.6 Campionamento

Teorema di Nyquist-Shannon

Il teorema di Nyquist-Shannon afferma che un segnale continuo può essere ricostruito perfettamente dai suoi campioni discreti **senza aliasing** se e solo se:

$$f_s \geq 2 f_{\max}$$

dove la frequenza minima di campionamento $2f_{\max}$ è chiamata **frequenza di Nyquist**.

Applicazione alle piramidi Gaussiane

La sfocatura Gaussiana (filtraggio passa-basso) riduce $f_{\max}$, prevenendo così l'aliasing quando l'immagine viene successivamente sottocampionata.

4.7 Immagini ibride

Le **immagini ibride** sono create combinando le componenti a bassa frequenza di un'immagine con le componenti ad alta frequenza di un'altra.

1. Un **filtro passa-basso** (es. Gaussiano) viene applicato alla prima immagine,

conservandone solo la struttura grossolana.

2. Un **filtro passa-alto** (es. Laplaciano) viene applicato alla seconda immagine, conservandone solo i dettagli fini e i bordi.
3. Le due immagini risultanti vengono sommate.

L'immagine ibrida finale viene percepita diversamente a seconda della distanza di visione: da lontano il sistema visivo coglie l'immagine a bassa frequenza; da vicino i dettagli ad alta frequenza diventano dominanti. Questo sfrutta il comportamento **dipendente dalla frequenza della visione umana**.

Chapter 5

Feature locali

5.1 Introduzione alle feature locali

Feature

Una **feature** è un'informazione sul contenuto di un'immagine. Si distinguono due tipi principali:

- **Feature globali**: descrivono un'immagine nel suo insieme per rappresentare un intero oggetto. Usate per il rilevamento di oggetti (l'oggetto è presente, sì/no?).
- **Feature locali**: descrivono patch o keypoint all'interno di un oggetto. Usate per il riconoscimento di oggetti, dove è necessario trovare corrispondenze tra parti degli oggetti.

Proprietà di buone feature locali

Buone feature locali devono essere **numerose** per il matching e **distintive** per una corrispondenza non ambigua. Crucialmente, devono essere **invarianti** alle trasformazioni fotometriche e geometriche.

Le feature locali sono utili perché sono: **locali** (robuste a occlusione parziale e cluttering), **numerose**, **distintive** (permettono un matching non ambiguo) ed **efficienti** (adatte ad applicazioni in tempo reale).

5.2 Invarianza ed equivarianza

- **Invarianza**: una trasformazione dell'immagine non cambia la proprietà della feature rilevata.
- **Equivarianza**: se l'immagine è trasformata geometricamente, le posizioni delle feature rilevate si trasformano di conseguenza, così i punti corrispondenti vengono ancora rilevati in posizioni corrispondenti.

Per i rilevatori di feature locali, si vuole tipicamente **invarianza** alle trasformazioni fotometriche ed **equivarianza** alle trasformazioni geometriche.

5.3 Applicazioni delle feature locali

Feature locali sono usate in: allineamento di immagini e stitching di panorami, ricostruzione 3D, SLAM visivo, tracking in realtà aumentata, riconoscimento di oggetti, image retrieval e navigazione robotica/veicolare.

5.4 Detection, description e matching

I metodi a feature locali sono tipicamente composti da tre fasi:

- **Detection:** identificare punti di interesse o keypoint nell'immagine.
- **Description:** estrarre un vettore descrittore dalla patch intorno a ogni keypoint.
- **Matching:** confrontare i descrittori tra immagini diverse per trovare corrispondenze.

5.5 Rilevatore di angoli di Harris

Harris corner detector

Il *rilevatore di angoli di Harris* identifica punti importanti traslando una piccola finestra intorno a essi e verificando dove si verificano forti cambiamenti di aspetto.

5.5.1 La matematica del rilevamento di angoli

Per quantificare il cambiamento di intensità, si calcola la **Sum of Squared Differences (SSD)** per una finestra W traslata di un piccolo offset (u, v) :

$$E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$$

Un angolo è un punto in cui questo errore $E(u, v)$ è elevato per tutti i piccoli spostamenti (u, v) .

Approssimazione per piccoli spostamenti. Per piccoli spostamenti si approssima il cambiamento di intensità con un'espansione di Taylor al primo ordine:

$$I(x+u, y+v) \approx I(x, y) + I_x u + I_y v$$

Sostituendo nella formula della SSD:

$$E(u, v) \approx \sum_{(x,y) \in W} [I_x u + I_y v]^2 \approx Au^2 + 2Buv + Cv^2$$

dove $A = \sum I_x^2$, $B = \sum I_x I_y$, $C = \sum I_y^2$ (somme sulla finestra W).

5.5.2 La matrice dei secondi momenti

La forma quadratica si esprime in notazione matriciale:

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} u \\ v \end{bmatrix}$$

La matrice $\mathbf{H}$ è la **matrice dei secondi momenti** (o *structure tensor*):

$$\mathbf{H} = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

5.5.3 Interpretazione della matrice dei secondi momenti

La forma della superficie di errore quadratica $E(u, v)$ è determinata dagli autovalori λ_1 e λ_2 di $\mathbf{H}$:

- Gli **autovettori** di $\mathbf{H}$ corrispondono alle direzioni di massimo e minimo cambiamento di intensità.
- Gli **autovalori** $\lambda_{\max}$ e $\lambda_{\min}$ rappresentano la magnitudine di tale cambiamento in quelle direzioni.

Questo permette di classificare la regione:

- **Flat**: entrambi λ_1 e λ_2 piccoli. Errore basso in tutte le direzioni.
- **Edge**: un autovalore grande e l'altro piccolo ($\lambda_1 \gg \lambda_2$). Errore alto in una direzione ma basso lungo il bordo.
- **Corner**: entrambi λ_1 e λ_2 grandi e di magnitudine simile. Errore alto in tutte le direzioni.

Criterio del minimo autovalore

Una misura semplice di *cornerness* è $\lambda_{\min}$, il minore autovalore della matrice dei secondi momenti. Un punto è considerato un buon angolo se $\lambda_{\min}$ supera una soglia.

5.5.4 L'operatore di Harris e l'algoritmo

Calcolare gli autovalori per ogni pixel è inefficiente. Il rilevatore di Harris usa una **funzione di risposta** R che approssima il comportamento degli autovalori senza calcolarli esplicitamente:

$$R = \det(\mathbf{H}) - k \cdot \text{trace}^2(\mathbf{H}) = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

dove k è una costante empirica (tipicamente 0.04–0.06). Il valore di R indica la *cornerness*: grande e positivo per un angolo, grande e negativo per un bordo, piccolo per una regione piatta.

Algoritmo del rilevatore di Harris:

1. Calcolare le derivate dell'immagine I_x e I_y (es. usando filtri Sobel).
2. Per ogni pixel, calcolare la matrice dei secondi momenti $\mathbf{H}$ su una finestra Gaussiana intorno ad esso.
3. Calcolare la risposta di Harris R per ogni pixel.
4. Sogliare la mappa di risposta R per identificare candidati angoli forti.
5. Eseguire la **non-maximum suppression** per trovare i punti angolo finali.

5.5.5 Proprietà e limitazioni

Il rilevatore di Harris è:

- **Equivariante** alla rotazione (l'ellisse dei secondi momenti ruota, ma gli autovalori sono preservati) e alla traslazione.
- **Invariante** agli shift di luminosità ($+b$) ma solo parzialmente ai cambiamenti di contrasto ($\times a$), poiché la sua risposta R scala, influenzando la sogliatura.
- **Non invariante alla scala**: un angolo può apparire piatto o come un bordo quando si esegue lo zoom.

5.6 Rilevamento di blob invariante alla scala

Per superare la limitazione di scala di Harris, i *blob* invarianti alla scala vengono rilevati trovando la loro **scala caratteristica**: la scala σ in uno **scale-space 3D** (x, y, σ) in cui una funzione di risposta è massimizzata.

Il **LoG normalizzato alla scala** è un comune rilevatore di blob:

$$\nabla_{\text{norm}}^2 g = \sigma^2 \left(\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right)$$

Massimizzare questa risposta LoG attraverso le scale fornisce la scala caratteristica.

Scale-space extrema

Il rilevamento invariante alla scala può essere formulato come ricerca dei massimi locali in uno scale-space 3D (x, y, σ) . Un keypoint viene selezionato non solo se la sua risposta è massima localmente nel piano immagine, ma anche se è massima tra le scale vicine. Il fattore σ^2 nella LoG normalizzata è necessario per rendere le risposte comparabili tra scale diverse.

Implementazione efficiente

Invece di applicare filtri sempre più grandi all'immagine, è più efficiente costruire una piramide Gaussiana e applicare un filtro di dimensione fissa a ogni livello:

1. Costruire uno scale-space calcolando la risposta LoG per un range di scale σ .
2. Trovare i massimi locali nello spazio 3D posizione-scala (x, y, σ) .
3. La posizione (x, y) e la scala caratteristica σ di ogni massimo definiscono una feature rilevata.

Filtro LoG a dimensione fissa e scala effettiva

Quando si usa una piramide Gaussiana per il rilevamento di blob invariante alla scala con un filtro LoG a dimensione fissa e σ interno fisso:

- **Livello 0** (immagine originale): scala effettiva $\approx \sigma_{\text{filter}}$.
- **Livello 1** (sottocampionato $2\times$): ogni pixel del filtro copre 2×2 pixel dell'immagine originale; scala effettiva $\approx 2 \times \sigma_{\text{filter}}$.
- **Livello 2** (sottocampionato $4\times$): scala effettiva $\approx 4 \times \sigma_{\text{filter}}$.

La scala caratteristica per un blob rilevato è la scala del livello piramidale in cui la sua risposta LoG è massimizzata.

Chapter 6

Descrittori locali

6.1 Progettare descrittori di feature

Descrittore di feature

Un **descrittore di feature** è una **rappresentazione vettoriale** della patch che circonda un keypoint. L'obiettivo è progettare descrittori tali che **patch con contenuto simile abbiano descrittori simili**, abilitando un matching robusto. Un buon descrittore deve essere robusto alle trasformazioni fotometriche e geometriche: il design è un compromesso tra **distintività** e **invarianza**.

6.1.1 Evoluzione di un descrittore

1. **Patch grezza**: intensità dei pixel; perfetta per match esatti ma altamente sensibile a variazioni di luminosità e posizione.
2. **Gradienti dell'immagine**: invarianti agli shift di luminosità ma sensibili a deformazione, rotazione e cambiamenti di contrasto.
3. **Istogramma colore**: invariante a rotazione/scala ma perde tutto il layout spaziale, rendendolo non discriminativo.
4. **Istogrammi spaziali**: istogrammi per cella (come in HOG o nelle regioni di SIFT). Parzialmente invarianti alla deformazione; sensibili alla rotazione a meno che non si stabilisca prima un'orientazione dominante.
5. **Normalizzazione dell'orientazione**: raggiunge la piena invarianza alla rotazione allineando la patch alla sua orientazione dominante prima del calcolo del descrittore (come in SIFT).

6.1.2 Confronto di istogrammi

- **Intersezione**: $\cap(Q, V) = \sum_i \min(q_i, v_i)$. Robusta.
- **Distanza Euclidea**: $d(Q, V) = \sqrt{\sum_i (q_i - v_i)^2}$. Meno robusta: pesa tutti i bin ugualmente.
- **Chi-quadro**: $\chi^2(Q, V) = \sum_i \frac{(q_i - v_i)^2}{q_i + v_i}$. Più discriminativa: pesa i bin diversa-

mente.

6.2 Descrittore HOG (Histogram of Oriented Gradients)

HOG

HOG è un potente descrittore di feature usato per il rilevamento di pedoni, che combina gradiente dell'immagine, celle spaziali e normalizzazione locale del contrasto.

Il descrittore viene calcolato su una patch di dimensione fissa (es. 64×128 pixel per il rilevamento di pedoni).

1. Calcolo del gradiente. Si calcolano i gradienti orizzontale e verticale per ottenere magnitudine e direzione per ogni pixel, usando **gradienti unsigned** con angoli da 0° a 180° (direzioni opposte trattate ugualmente).

2. Istogrammi per cella. La patch viene divisa in piccole regioni dette **celle** (es. 8×8 pixel). Per ogni cella si crea un istogramma delle orientazioni del gradiente con **9 bin** (per angoli $0^\circ, 20^\circ, \dots, 160^\circ$). Ogni pixel vota con peso uguale alla sua magnitudine, nel bin corrispondente alla sua orientazione. I voti vengono interpolati bilinearmente tra bin adiacenti per evitare aliasing.

3. Normalizzazione per blocco. Le celle adiacenti vengono raggruppate in **blocchi** (es. 2×2 celle, 16×16 pixel). Gli istogrammi di tutte le celle nel blocco vengono concatenati e normalizzati:

$$v_{\text{norm}} = \frac{v}{\sqrt{\|v\|_2^2 + \varepsilon}}$$

Questo rende il descrittore **robusto ai cambiamenti fotometrici locali**.

4. Vettore di feature. Tutti i vettori di blocco normalizzati vengono concatenati nel descrittore HOG finale. I blocchi si sovrappongono (finestra mossa di 8 pixel alla volta), fornendo una rappresentazione ridondante e robusta. Le celle d'angolo vengono codificate una volta, quelle di bordo due volte, quelle interne quattro volte.

HOG per il rilevamento di pedoni

Per una patch 64×128 con celle 8×8 e blocchi 2×2 (step 8 pixel), si ottengono $7 \times 15 = 105$ blocchi, ciascuno con $4 \times 9 = 36$ valori. Il descrittore HOG finale è un **vettore a 3780 dimensioni**:

$$7 \times 15 \times 36 = 3780$$

Questo vettore può essere alimentato in un classificatore lineare (es. SVM). La visualizzazione del descrittore HOG cattura chiaramente la forma della persona, specialmente intorno al tronco e alle gambe.

Proprietà di HOG

HOG standard non ha assegnazione di orientazione dominante: è adatto a oggetti con posa canonica (pedoni in piedi) e **non è invariante alla rotazione**. È robusto ai cambiamenti locali di illuminazione grazie alla normalizzazione per blocco, ma non invariante alla scala.

Chapter 7

Scale-Invariant Feature Transform (SIFT)

Il *Scale-Invariant Feature Transform* (**SIFT**) è un algoritmo di riferimento per **rilevare** e **descrivere** feature locali nelle immagini, progettato per essere robusto a variazioni di scala, rotazione e illuminazione. SIFT è sia un rilevatore che un descrittore; ogni keypoint è rappresentato come $(x, y, s, \theta, \mathbf{d})$.

7.1 Rilevatore SIFT

7.1.1 Costruzione dello scale-space

SIFT usa una piramide **Difference of Gaussians (DoG)**, approssimazione efficiente del Laplaciano del Gaussiano (LoG):

$$\text{DoG}(x, y, \sigma) = G(x, y, k\sigma) - G(x, y, \sigma)$$

Sia DoG che LoG sono equivarianti alla rotazione.

Una piramide Gaussiana viene costruita in **ottave**. Ogni ottava contiene immagini della stessa risoluzione, progressivamente sfocate. Per passare all'ottava successiva si sottocampiona di un fattore 2. Le immagini DoG si calcolano sottraendo livelli adiacenti all'interno di ogni ottava.

7.1.2 Rilevamento degli estremi nello scale-space

Un pixel è un keypoint potenziale se è un estremo locale rispetto ai suoi **8 vicini spaziali** nella stessa immagine DoG e ai **9 vicini** nelle immagini DoG sopra e sotto, per un totale di **26 vicini**.

Rimozione dei keypoint instabili:

- **Basso contrasto:** risposta DoG debole, sensibile al rumore. Si scartano keypoint con risposta sotto una soglia.
- **Su bordi:** localizzazione ambigua lungo la direzione del bordo. La risposta DoG cambia fortemente in una sola direzione (trasversale al bordo) $\Rightarrow$ non dis-

tinti per il matching. Su un buon keypoint (blob o angolo) la risposta cambia significativamente in tutte le direzioni.

Eliminazione dei keypoint sui bordi (Hessian Ratio Test)

Per scartare i punti di interesse situati lungo i bordi rettilinei (dove la localizzazione è ambigua lungo la direzione del bordo), SIFT calcola la matrice Hessiana 2D $\mathbf{H}$ della risposta DoG nel punto candidato:

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

Il rapporto tra i suoi autovalori α, β (curvature principali) viene verificato calcolando la traccia e il determinante senza calcolare gli autovalori espliciti:

$$\frac{\text{trace}(\mathbf{H})^2}{\det(\mathbf{H})} = \frac{(\alpha + \beta)^2}{\alpha\beta} = \frac{(r + 1)^2}{r}$$

Se $\frac{\text{trace}(\mathbf{H})^2}{\det(\mathbf{H})} \geq \frac{(r+1)^2}{r}$ (con soglia tipica $r = 10$), significa che il punto si trova su un bordo e viene eliminato.

7.1.3 Assegnazione dell'orientazione

Per garantire l'invarianza alla rotazione si assegna un'**orientazione canonica** a ogni keypoint:

1. **Gradiente del vicinato:** si calcolano magnitudine e orientazione per i pixel nell'intorno del keypoint nell'immagine Gaussiana corrispondente.
2. **Istogramma di orientazione:** istogramma a **36 bin** (360° , bin da 10°); ogni pixel vota pesato dalla magnitudine e da una finestra Gaussiana centrata sul keypoint.
3. **Orientazione/i dominante/i:** il picco più alto definisce l'orientazione primaria. Qualsiasi picco superiore all'**80%** del principale genera un nuovo keypoint con stessa posizione e scala.

7.2 Descrittore SIFT

Assegnati posizione (x, y) , scala σ e orientazione θ al keypoint:

1. Si prende una finestra 16×16 dall'immagine Gaussiana sfocata alla scala caratteristica.
2. Si **ruotano** coordinate e gradienti secondo l'orientazione canonica $\Rightarrow$ **invarianza alla rotazione**.
3. La finestra 16×16 viene divisa in una griglia 4×4 di celle (4×4 pixel ciascuna, 16 celle in totale). Per ogni cella si calcola un **istogramma di orientazione a 8 bin**.
4. Si **concatenano** i 16 istogrammi in un vettore a **128 dimensioni** ($16 \times 8 = 128$).

5. Si **normalizza** a lunghezza unitaria $\Rightarrow$ robustezza ai cambiamenti di illuminazione.

Proprietà di SIFT

Il descrittore SIFT è invariante a:

- **Scala:** rilevamento in piramide di scale-space.
- **Rotazione:** normalizzazione dell'orientazione.
- **Illuminazione:** normalizzazione del vettore descrittore.

È anche robusto alle trasformazioni affini e al rumore, ma non completamente invariante a grandi cambiamenti di punto di vista.

7.3 Matching di feature

7.3.1 Ratio test di Lowe

Per ogni feature f_1 in I_1 si trovano i due vicini più prossimi in I_2 : f_2 (best match) e f'_2 (second-best). Si calcola il ratio:

$$\text{ratio} = \frac{\|f_1 - f_2\|}{\|f_1 - f'_2\|}$$

Il match è accettato se $\text{ratio} < \text{soglia}$ (tipicamente 0.75–0.8). Un ratio piccolo indica un match non ambiguo: il miglior match è significativamente migliore del secondo migliore.

7.4 Valutazione del matching

Variando la soglia si ottiene la **ROC curve**:

$$\text{TPR} = \frac{\#\text{TP}}{\#\text{match corretti totali}}, \quad \text{FPR} = \frac{\#\text{FP}}{\#\text{match scorretti totali}}$$

Metriche informative

La semplice accuracy è scadente per il feature matching: i veri negativi (match non corrispondenti) sono astronomicamente numerosi e dominano il calcolo. Preferire:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{P \cdot R}{P + R}$$

L'**AUC** (Area Under the ROC Curve) riassume la performance complessiva: 1.0 = classificatore perfetto, 0.5 = random.

7.5 SURF: Speeded-Up Robust Features

SURF è un'alternativa più veloce a SIFT con qualità comparabile.

Rilevamento (Hessiano): SURF rileva blob dove $\det(\mathbf{H})$ è massimo, con:

$$\mathbf{H}(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx} & L_{xy} \\ L_{xy} & L_{yy} \end{bmatrix}, \quad \det(H_{\text{approx}}) = D_{xx}D_{yy} - (wD_{xy})^2, \quad w = 0.9$$

Le derivate Gaussianhe del secondo ordine vengono approssimate con **filtri box** calcolati tramite **immagini integrali** per massima efficienza.

Descrittore: finestra $20s \times 20s$ allineata all'orientazione dominante, divisa in griglia 4×4 . Per ogni cella: $(\sum dx, \sum dy, \sum |dx|, \sum |dy|) \Rightarrow$ **vettore a 64 dimensioni**.

7.6 Descrittori binari di feature

Alternative meno costose a SIFT che generano **stringhe binarie brevi**, molto veloci da calcolare e confrontare.

Per ogni coppia di pixel (s_1, s_2) nella patch:

$$b = \begin{cases} 1 & \text{se } I(s_1) < I(s_2) \\ 0 & \text{altrimenti} \end{cases}$$

Vantaggi

Compatti (es. 256 bit per ORB); veloci da calcolare (semplici confronti di intensità); veloci da confrontare tramite **distanza di Hamming**: $d_H(B_1, B_2) = \text{sum}(\text{xor}(B_1, B_2))$.

ORB (Oriented FAST and Rotated BRIEF): migliora BRIEF aggiungendo invarianza alla rotazione. Stima il centro di massa (CoM) e ruota di θ le 256 coppie apprese da training (coppie non correlate ad alta varianza) prima di eseguire i confronti.

7.7 Deep local features

SuperPoint apprende detection e description tramite dati sintetici e self-supervised learning. **SuperGlue** apprende il matching con graph neural networks.

Chapter 8

Trasformazioni e allineamento

8.1 Image warping

- **Image filtering:** cambia i valori dei pixel (range) $\Rightarrow g(x) = h(f(x))$.
- **Image warping:** cambia le coordinate dei pixel (dominio) $\Rightarrow g(x) = f(h(x))$, dove h è una trasformazione di coordinate.

Ci si concentra sul **global warping**: una singola trasformazione $\mathbf{p}' = T(\mathbf{p})$ applicata a tutti i punti $\mathbf{p} = (x, y)$.

8.2 Tipi di trasformazioni 2D

Trasformazione	DOF	Min. corrisp.	Preserva
Traslazione	2	1	orientazione, lunghezze, angoli, parallelismo
Euclidea/rigid	3	—	lunghezze, angoli, parallelismo
Similarità	4	—	angoli, parallelismo, rapporti
Affine	6	3	linee, parallelismo, rapporti su parallele
Proiettiva	8	4	linee rette

8.2.1 Trasformazioni lineari

$$\mathbf{p}' = \mathbf{T}\mathbf{p} \implies \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Esempi:

- Scala: $\mathbf{S} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$
- Rotazione: $\mathbf{R} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$
- Shear: $\mathbf{Sh} = \begin{bmatrix} 1 & sh_x \\ sh_y & 1 \end{bmatrix}$

Proprietà

L'origine mappa nell'origine; le linee mappano in linee; il parallelismo è preservato; i rapporti di lunghezze lungo una linea sono preservati; chiuse sotto composizione.

Limitazione delle matrici 2×2

La traslazione non è lineare su coordinate 2D e non può essere rappresentata da una matrice 2×2 .

8.2.2 Trasformazioni affini

Con **coordinate omogenee** $(x, y) \rightarrow (x, y, 1)$ si includono le traslazioni:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Da coordinate omogenee a 2D

$(x, y, w) \rightarrow (x/w, y/w)$ (normalizzazione). I punti con $w = 0$ sono punti all'infinito (utili per i punti di fuga).

La trasformazione affine mappa linee in linee, preserva il parallelismo e i rapporti lungo linee parallele. L'origine non mappa necessariamente nell'origine.

8.2.3 Trasformazioni proiettive (omografie)

Matrice 3×3 **H** definita a meno di un fattore di scala (8 DOF):

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \underbrace{\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} x \\ y \\ w \end{bmatrix}, \quad \text{coordinate Cartesiane: } (x'/w', y'/w')$$

Proprietà

Le linee rette mappano in linee rette. Le linee parallele non rimangono necessariamente parallele (convergono a punti di fuga). I rapporti non sono preservati.

Applicazione

Un'omografia mappa punti tra due immagini diverse dello stesso piano 3D. È la trasformazione che relaziona due viste della stessa superficie planare nel modello a camera pinhole.

8.3 Calcolo delle trasformazioni

8.3.1 Sistema lineare e minimi quadrati

Con più della minima quantità di corrispondenze si ottiene un sistema **sovradeterminato** $\mathbf{A}\mathbf{t} = \mathbf{b}$, risolto con i **minimi quadrati lineari**:

$$\mathbf{t} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b} = \mathbf{A}^\# \mathbf{b}$$

8.3.2 Calcolo di un'omografia

Per una corrispondenza $(x_i, y_i) \rightarrow (x'_i, y'_i)$, riorganizzando si ottiene $\mathbf{A}\mathbf{h} = \mathbf{0}$ con due righe per corrispondenza:

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i x_i & -y'_i y_i & -y'_i \end{bmatrix} \mathbf{h} = \mathbf{0}$$

Con 4 corrispondenze (8 equazioni) si risolve il sistema omogeneo: minimizzare $\|\mathbf{A}\mathbf{h}\|^2$ soggetto a $\|\mathbf{h}\| = 1$.

Soluzione SVD

$\mathbf{h}$ è l'autovettore di $\mathbf{A}^T \mathbf{A}$ per il minimo autovalore. Si usa la SVD di $\mathbf{A}$: $\mathbf{h}$ è l'ultima colonna di $\mathbf{V}$.

8.4 Stima robusta con RANSAC

I minimi quadrati sono sensibili agli **outlier** (match scorretti). **RANSAC** (RANDOM SAmple Consensus) identifica il più grande insieme di inlier:

1. **Campiona** s punti (2 per lineare, 3 per affine, 4 per omografia).
2. **Calcola** il modello dal campione minimale.
3. **Conta** gli inlier entro tolleranza ε .
4. **Ripeti** per N iterazioni; conserva il modello con più inlier.
5. **Ricalcola** il modello con tutti gli inlier tramite minimi quadrati.

Numero teorico di iterazioni in RANSAC

Il numero minimo di iterazioni N necessario a RANSAC per garantire con probabilità p (es. 99%) di pescare almeno un sottoinsieme di s punti composto esclusivamente da inlier (con un tasso stimato di outlier e) è:

$$N \geq \frac{\log(1-p)}{\log(1-(1-e)^s)}$$

Implicazione d'esame: All'aumentare della percentuale di outlier ($e \rightarrow 1$), il denominatore tende a zero e il numero di iterazioni N necessarie cresce in modo **esponenziale**.

8.5 Implementazione del warping e blending

8.5.1 Forward vs. inverse warping

- **Forward:** per ogni pixel (x, y) sorgente si calcola $(x', y') = T(x, y)$ e si copia il valore. *Problema:* buchi e sovrapposizioni nell'output.
- **Inverse:** per ogni pixel (x', y') destinazione si calcola $(x, y) = T^{-1}(x', y')$ e si campiona il colore. **Garantisce ogni pixel riempito.** Poiché (x, y) non è in genere intero, serve **interpolazione** (nearest neighbor, bilineare, bicubica).

8.5.2 Blending

- **Feathering:** media pesata (alpha blending) con transizione graduale nelle zone sovrapposte. Può produrre ghosting se l'allineamento è imperfetto.
- **Pyramid blending:** si veda Capitolo 3.
- **Poisson blending:** metodo nel dominio del gradiente che risolve un'equazione di Poisson per preservare i gradienti nella regione incollata.

8.6 Punti di fuga

Un **punto di fuga** è la proiezione di una direzione 3D. Le linee 3D parallele con la stessa direzione convergono in esso. Per due rette l_1, l_2 in coordinate omogenee: $v = l_1 \times l_2$.

Chapter 9

Riconoscimento

9.1 Compiti di riconoscimento

Compito	Output
Image classification	Un'etichetta di classe per l'intera immagine
Semantic segmentation	Un'etichetta semantica per ogni pixel
Object detection	Bounding box ed etichette di classe
Instance segmentation	Etichetta semantica e identità di istanza per i pixel degli oggetti

9.2 Classificazione di immagini

Si usano approcci di ML con dataset etichettati (es. ImageNet). Il **transfer learning** consente il pre-training su un dataset grande e il fine-tuning su uno più piccolo.

Baseline nearest neighbor: si assegna la classe dell'immagine di training più vicina. Le distanze pixel-wise sono però poco informative a causa di shift, illuminazione o deformazioni.

9.2.1 Bag-of-Words (BoW)

BoW raggiunge l'**invarianza spaziale** rappresentando immagini come istogrammi di feature locali, ignorando la loro disposizione spaziale.

Pipeline:

1. Rilevare e descrivere feature (es. SIFT).
2. Raggruppare feature con k-means per formare il vocabolario visivo.
3. Assegnare ogni descrittore alla parola visiva più vicina.
4. Rappresentare ogni immagine con l'istogramma di frequenza delle parole.
5. Addestrare un classificatore (es. SVM) sugli istogrammi.

9.3 Reti Neurali Convoluzionali (CNN)

Tre tipi principali di strati:

- **Convoluzionali**: filtri apprendibili con **weight sharing**. Equivarianti alla traslazione.
- **Pooling**: riducono la risoluzione spaziale e aumentano il **campo ricettivo**. Solitamente senza parametri apprendibili.
- **FC**: classificazione finale.

9.3.1 Output e funzione di loss

Funzione **softmax**:

$$p_c = \frac{e^{s_c}}{\sum_j e^{s_j}}$$

Loss **cross-entropy**:

$$L = -\log p_y$$

9.3.2 Architetture principali

Architettura	Idea chiave
LeNet-5	Prima CNN per cifre
AlexNet	ImageNet breakthrough, ReLU, dropout, data augmentation
VGG	Stack profondi di conv. 3×3
GoogLeNet/Inception	Moduli convoluzionali multi-scala, meno parametri
ResNet	Connessioni residuali per reti molto profonde

ResNet mitiga il **vanishing gradient** con connessioni residuali che permettono al gradiente di bypassare gli strati.

9.4 Segmentazione semantica

Usa l'**architettura encoder-decoder**:

- *Encoder*: downsampling progressivo per feature di alto livello.
- *Decoder*: upsampling progressivo per produrre la mappa di segmentazione.
- **Skip connections**: combinano informazioni grossolane del decoder con quelle fini dell'encoder.

Metriche: IoU (Jaccard Index) = $\frac{|P \cap G|}{|P \cup G|}$, accuratezza per pixel.

Fully Convolutional Networks (FCN): sostituiscono i layer FC con operazioni convoluzionali producendo heatmap spaziali. Skip connections recuperano il dettaglio spaziale.

SegNet: encoder-decoder con pooling indices per guidare l'unpooling.

Convoluzione dilatata: aumenta il campo ricettivo senza aumentare i parametri e senza ridurre la risoluzione spaziale. Utile per predizione densa.

U-Net: encoder-decoder con skip connections tra livelli corrispondenti. Le skip connections concatenano feature a bassa e alta risoluzione, migliorando la localizzazione. Standard de facto per segmentazione biomedicale.

9.5 Rilevamento di oggetti

mAP (mean Average Precision): metrica principale calcolata dalle curve precisione-recall, con soglia IoU ≥ 0.5 per i veri positivi.

9.5.1 Rilevatori a due stadi

- **R-CNN (2014):** Selective Search per $\sim 2k$ proposte; CNN su ciascuna indipendentemente. Lento.
- **Fast R-CNN (2015):** CNN sull'intera immagine; **RoI Pooling** per feature a dimensione fissa da ogni proposta.
- **Faster R-CNN (2015): Region Proposal Network (RPN)** apprendibile; predice objectness e raffinamenti rispetto ad **anchor box** predefiniti. Molto più veloce.

9.5.2 Rilevatori a stadio singolo

YOLO e **SSD** predicono bounding box e probabilità di classe direttamente in un singolo passaggio. Generalmente più veloci.

9.5.3 Feature Pyramid Networks (FPN)

Costruiscono una **piramide di feature multi-scala** con percorso top-down e connessioni laterali, permettendo il rilevamento a multiple scale.

9.5.4 Sliding window e HOG

Approccio classico: finestra scorrevole + HOG + classificatore SVM. Non-maximum suppression rimuove i rilevamenti duplicati. Semplice ma lento.

9.6 Estensioni

Mask R-CNN: estende Faster R-CNN con un ramo di segmentazione per ogni oggetto rilevato. Sostituisce RoI Pooling con **RoIAlign** per preservare l'allineamento spaziale.

RoIPooling vs. RoIAlign: Il problema della Quantizzazione

In Faster R-CNN, il **RoIPooling** esegue un arrotondamento (quantizzazione) sulle coordinate decimali della regione d'interesse per adattarle alla griglia discreta dei pixel della feature map. Questo errore di disallineamento spaziale (≈ 0.5 pixel) è trascurabile per le Bounding Box, ma distruttivo per le maschere di segmentazione al pixel.

Mask R-CNN sostituisce RoIPooling con **RoIAlign**: evita qualsiasi quantizzazione e calcola il valore esatto delle caratteristiche in punti a coordinate decimali utilizzando l'**interpolazione bilineare** a partire da 4 punti di campionamento, preservando un perfetto allineamento spaziale.

Segmentazione panoptica: unifica segmentazione semantica (“stuff”: cielo, strada) e di istanze (“things”: auto, persone).

Chapter 10

Flusso ottico

10.1 Introduzione

Flusso ottico

Il **flusso ottico** è il moto apparente di oggetti, superfici e bordi in una scena visiva causato dal moto relativo tra osservatore e scena. È un **campo vettoriale 2D** dove ogni vettore è uno spostamento di pixel tra fotogrammi successivi.

Visualizzazione tipica: codifica HSV, dove la tonalità indica la direzione e la luminosità la magnitudine.

Campo di moto vs. flusso ottico

- **Campo di moto**: vera proiezione 2D del moto 3D sul piano immagine.
- **Flusso ottico**: moto apparente dei pattern di luminosità. Possono differire: sfera Lambertiana ruotante sotto illuminazione fissa → campo di moto ma flusso ottico nullo; sfera speculare ferma con sorgente di luce mobile → flusso ottico ma campo di moto nullo.

Stereo vs. flusso ottico

Stereo	Flusso ottico
Due immagini allo stesso tempo	Due immagini in tempi diversi
Ricerca 1D lungo epipolare	Stima moto 2D (tutte le direzioni)
Usa geometria epipolare	Nessun vincolo epipolare diretto

10.2 Il problema dell'apertura

Osservando un bordo in movimento in una piccola patch locale, solo il moto perpendicolare al bordo può essere determinato (**flusso normale**); la componente parallela rimane ambigua. Per evitare il problema si necessita di patch con gradienti in direzioni diverse (angoli, texture).

10.3 Stima del flusso ottico

Assunzioni:

- **Costanza della luminosità:** $I(x(t), y(t), t)$ costante nel tempo.
- **Piccolo moto:** i punti non si spostano troppo tra fotogrammi.
- **Coerenza spaziale:** i punti vicini si muovono in modo simile.

10.3.1 Equazione di costanza della luminosità

Dalla costanza della luminosità e dall'espansione di Taylor al primo ordine:

$$I_x u + I_y v + I_t = 0$$

dove I_x, I_y sono i gradienti spaziali e I_t è il gradiente temporale.

Problema sottodeterminato

Un'equazione per pixel, due incognite (u, v) : le soluzioni giacciono su una retta nello spazio $(u, v) \Rightarrow$ problema sottodeterminato per singoli pixel (apertura problem).

10.4 Lucas-Kanade

Assume il flusso **costante** in una piccola patch $N \times N$. Sistema sovradeterminato $\mathbf{Ax} = \mathbf{b}$:

$$\underbrace{\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{N^2}) & I_y(\mathbf{p}_{N^2}) \end{bmatrix}}_{\mathbf{A}} \begin{bmatrix} u \\ v \end{bmatrix} = \underbrace{\begin{bmatrix} -I_t(\mathbf{p}_1) \\ \vdots \\ -I_t(\mathbf{p}_{N^2}) \end{bmatrix}}_{\mathbf{b}}$$

Soluzione minimi quadrati: $\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$.

La matrice $\mathbf{A}^T \mathbf{A}$ è la **matrice dei secondi momenti** (come in Harris):

$$\mathbf{A}^T \mathbf{A} = \sum_{\mathbf{p}} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Quando funziona Lucas-Kanade?

Regione	Autovalori λ_1, λ_2	Flusso
Piatta	Entrambi piccoli	Non affidabile
Bordo	Uno grande, uno piccolo	Ambiguo
Angolo/texture	Entrambi grandi	Affidabile

10.4.1 Gestione dei grandi moti: coarse-to-fine

L'assunzione di piccolo moto fallisce per spostamenti grandi. Si usa la strategia **coarse-to-fine** con piramidi Gaussianhe:

1. Costruire piramidi Gaussiani per entrambi i fotogrammi.
2. Stimare $(u, v)^{(0)}$ al livello più grossolano (grandi moti appaiono piccoli).
3. Iterativamente: upsample del flusso $\rightarrow$ warp del fotogramma $t+1$ verso t (inverse warping) $\rightarrow$ calcolo del residuo $\Delta(u, v) \rightarrow$ accumulo.
4. Ripetere fino alla risoluzione piena.

10.5 Horn-Schunck

Assume il flusso **regolare** sull'intera immagine. Minimizza:

$$E(u, v) = \underbrace{\iint (I_x u + I_y v + I_t)^2 dx dy}_{\text{termine dati}} + \lambda \underbrace{\iint (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy}_{\text{regolarizzazione}}$$

Oversmoothing

La penalità quadratica penalizza troppo le discontinuità di flusso ai confini degli oggetti, causando **oversmoothing**. Si risolve con funzioni di loss **robuste** (es. penalità Student-t) meno sensibili ai grandi errori.

10.6 Deep learning per il flusso ottico

FlowNet: prima rete end-to-end supervisionata per flusso ottico. Architettura encoder-decoder su due fotogrammi consecutivi.

- FlowNetSimple: stack dei due frame in input a una singola rete.
- FlowNetCorr: due stream con correlation layer per il matching.

End-Point Error (EPE): $EPE = \sqrt{(u - \hat{u})^2 + (v - \hat{v})^2}$

FlowNet2: impila più moduli FlowNet con warping tra i moduli.

PWC-Net (Pyramid, Warping, Cost volume): piramide di feature, stima coarse-to-fine, warping delle feature del secondo frame verso il primo, cost volume per il matching. Combina principi classici e DL.

Flusso non supervisionato: apprendimento senza ground-truth tramite loss di ricostruzione fotometrica e di smoothness.

Chapter 11

Calibrazione della camera

11.1 Il modello della camera

Una camera esegue una **trasformazione proiettiva** che mappa il mondo 3D su un piano immagine 2D. La **calibrazione** è il processo di stima dei parametri di questa trasformazione.

11.1.1 Pinhole e lente sottile

Il modello più semplice è la **camera pinhole**: una piccola apertura lascia passare un solo raggio per punto, producendo un'immagine invertita senza sfocatura.

Trade-off del pinhole

Apertura piccola $\rightarrow$ meno blur ma effetti di diffrazione e poca luce (lunga esposizione). Formula approssimata per il diametro ideale: $d \approx 2\sqrt{f\lambda}$.

Legge della lente sottile:

$$\frac{1}{i} + \frac{1}{o} = \frac{1}{f}$$

dove o è la distanza oggetto, i la distanza immagine, f la lunghezza focale. Solo gli oggetti a una profondità specifica sono a fuoco.

11.1.2 Proiezione prospettica

Relazione tra punto 3D $\mathbf{x}_c = (x_c, y_c, z_c)$ nel frame camera e proiezione 2D (x_i, y_i) :

$$x_i = f \frac{x_c}{z_c}, \quad y_i = f \frac{y_c}{z_c}$$

11.1.3 Dal piano immagine ai pixel

$$u = f_x \frac{x_c}{z_c} + o_x, \quad v = f_y \frac{y_c}{z_c} + o_y$$

dove $f_x = m_x f$, $f_y = m_y f$ (lunghezze focali in pixel) e (o_x, o_y) è il punto principale.

11.1.4 La matrice della camera

$$\tilde{\mathbf{u}} = \mathbf{P}\tilde{\mathbf{x}}_w = \mathbf{K}[\mathbf{R}|\mathbf{t}]\tilde{\mathbf{x}}_w$$

Matrice intrinseca $\mathbf{K}$ (parametri intrinseci f_x, f_y, o_x, o_y):

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

Matrice estrinseca $[\mathbf{R}|\mathbf{t}]$ (parametri estrinseci):

$$[\mathbf{R}|\mathbf{t}] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}$$

Distorsione dell'obiettivo

Gli obiettivi reali introducono distorsioni radiali e tangenziali:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = (1 + \kappa_1 r^2 + \kappa_2 r^4) \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} 2\kappa_3 xy + \kappa_4(r^2 + 2x^2) \\ 2\kappa_4 xy + \kappa_3(r^2 + 2y^2) \end{pmatrix}$$

con $r^2 = x^2 + y^2$. Poi: $(u, v) = (f_x x' + c_x, f_y y' + c_y)$.

11.2 Calibrazione della camera

Si stima $\mathbf{P}$ osservando un oggetto di geometria nota (es. scacchiera) con almeno 6 coppie di punti 3D noti e corrispondenti punti immagine (11 incognite perché $\mathbf{P}$ è definita a meno di scala).

Espansione delle equazioni DLT per singolo punto

Ogni corrispondenza tra punto 3D del mondo $\mathbf{X}_i = (x_i, y_i, z_i, 1)^T$ e pixel 2D $\mathbf{u}_i = (u_i, v_i, 1)^T$ soddisfa $\mathbf{u}_i \sim \mathbf{P}\mathbf{X}_i$. Eliminando lo scalare omogeneo tramite divisione per la terza riga, si ricavano due equazioni lineari indipendenti per ciascun punto:

$$u_i(p_{31}x_i + p_{32}y_i + p_{33}z_i + p_{34}) - (p_{11}x_i + p_{12}y_i + p_{13}z_i + p_{14}) = 0$$

$$v_i(p_{31}x_i + p_{32}y_i + p_{33}z_i + p_{34}) - (p_{21}x_i + p_{22}y_i + p_{23}z_i + p_{24}) = 0$$

Impilando queste due righe per $N \geq 6$ punti, si compone la matrice dei dati $\mathbf{A}_{2N \times 12}$ per il sistema lineare omogeneo $\mathbf{A}\mathbf{p} = \mathbf{0}$.

Ogni corrispondenza dà 2 equazioni. Sistema omogeneo $\mathbf{A}\mathbf{p} = \mathbf{0}$: minimizzare $\|\mathbf{A}\mathbf{p}\|^2$ soggetto a $\|\mathbf{p}\| = 1$. Soluzione: autovettore di $\mathbf{A}^T\mathbf{A}$ per il minimo autovalore.

11.2.1 Estrazione dei parametri

Tramite **fattorizzazione RQ**:

$$\mathbf{P} = [\mathbf{M}|\mathbf{p}_4] \implies \mathbf{M} = \mathbf{K}\mathbf{R}, \mathbf{p}_4 = \mathbf{K}\mathbf{t} \implies \mathbf{K} = \mathbf{R}\mathbf{Q}(\mathbf{M}), \mathbf{R} = \mathbf{K}^{-1}\mathbf{M}, \mathbf{t} = \mathbf{K}^{-1}\mathbf{p}_4$$

11.3 Punti di fuga

Un **punto di fuga** è la proiezione di una direzione 3D. Le linee 3D parallele convergono in esso. Per due rette l_1, l_2 : $v = l_1 \times l_2$.

11.4 Visione stereo

Usa **due o più camere** per inferire profondità tramite triangolazione.

11.4.1 Profondità dalla disparità

Per stereo rettificato (camere parallele, baseline b):

$$z = \frac{b f_x}{d} = \frac{b f_x}{u_l - u_r}$$

Alta disparità $d \rightarrow$ punto vicino; bassa $\rightarrow$ lontano.

11.4.2 Matching stereo

Matching a finestra:

1. Patch intorno al pixel in I_l .
2. Confronto con patch lungo la stessa scanline in I_r .
3. Best match con SSD/SAD/NCC; $d = u_l - u_r$.

Trade-off della finestra

Finestra piccola: cattura dettagli fini, mappa rumorosa. Finestra grande: robusta al rumore, sfoca le discontinuità. Casi problematici: regioni senza texture, pattern ripetuti, superfici speculari, discontinuità di profondità.

11.4.3 Stereo come minimizzazione di energia

$$E(d) = \underbrace{\sum_{(x,y)} E_d(x, y, d(x, y))}_{\text{dati}} + \lambda \underbrace{\sum_{(p,q) \in \mathcal{E}} E_s(d_p, d_q)}_{\text{smoothness}}$$

Termini di smoothness: Potts ($\mathbf{1}[d_p \neq d_q]$) o L1 ($|d_p - d_q|$). Ottimizzazione: programmazione dinamica per scanline (ma streaking) o graph cuts (globale).

11.5 Stereo attivo e luce strutturata

Un proiettore proietta pattern di luce strutturata sulla scena, creando texture artificiale e semplificando le corrispondenze. Base di sensori come Kinect e Apple TrueDepth.

Chapter 12

Geometria epipolare

12.1 Introduzione

Geometria epipolare

La **geometria epipolare** mette in relazione le proiezioni 2D ($\mathbf{x}, \mathbf{x}'$) di un punto 3D $\mathbf{X}$ da due viste diverse, fornendo un vincolo geometrico per trovare corrispondenze e pose relative.

La geometria multi-vista relaziona tre componenti: moto della camera, corrispondenze tra immagini e struttura 3D.

- **Calibrazione / pose estimation**: da corrispondenze 3D-2D note.
- **Triangolazione**: da camera note e corrispondenze 2D-2D.
- **Structure from Motion / SLAM**: stima sia moto che struttura.

Back-projection: da una singola immagine, un pixel non determina un punto 3D unico ma un raggio retroproiettato. Serve una seconda vista.

Definizioni:

- **Baseline**: segmento tra i centri $\mathbf{O}, \mathbf{O}'$.
- **Piano epipolare**: piano definito da $\mathbf{X}, \mathbf{O}, \mathbf{O}'$.
- **Epipoli** ($\mathbf{e}, \mathbf{e}'$): intersezione della baseline con i piani immagine. $\mathbf{e}$ = proiezione di $\mathbf{O}'$, $\mathbf{e}'$ = proiezione di $\mathbf{O}$.
- **Linee epipolari** ($\mathbf{l}, \mathbf{l}'$): intersezione del piano epipolare con i piani immagine.

Vincolo epipolare: $\mathbf{x}'$ deve giacere su $\mathbf{l}' \Rightarrow$ riduce la ricerca di corrispondenze da 2D a 1D.

12.2 La matrice essenziale

Matrice essenziale

$\mathbf{E} \in \mathbb{R}^{3 \times 3}$ codifica il vincolo epipolare per camere **calibrate** ($\mathbf{K}$ nota):

$$\mathbf{x}'^T \mathbf{E} \mathbf{x} = 0$$

Linee: $\mathbf{l}' = \mathbf{E} \mathbf{x}$, $\mathbf{l} = \mathbf{E}^T \mathbf{x}'$. Epipoli: $\mathbf{E} \mathbf{e} = \mathbf{0}$, $\mathbf{e}'^T \mathbf{E} = \mathbf{0}$.

12.2.1 Derivazione

Con $\mathbf{x} = \mathbf{K}^{-1} \mathbf{p}$, $\mathbf{x}' = \mathbf{R}(\mathbf{x} - \mathbf{t})$. La coplanarità di $\mathbf{x}$, $\mathbf{t}$, $\mathbf{x} - \mathbf{t}$ implica:

$$(\mathbf{x} - \mathbf{t})^T (\mathbf{t} \times \mathbf{x}) = 0 \implies \mathbf{x}'^T \mathbf{R}[\mathbf{t}]_{\times} \mathbf{x} = 0 \implies \mathbf{E} = \mathbf{R}[\mathbf{t}]_{\times}$$

Proprietà di E

- Opera su coordinate normalizzate.
- Rango 2, $\det(\mathbf{E}) = 0$.
- 5 DOF (3 per $\mathbf{R}$, 2 per direzione di $\mathbf{t}$).
- 2 valori singolari non-zero uguali.

E vs. H

$\mathbf{E}$ mappa un punto in una linea ($\mathbf{l}' = \mathbf{E} \mathbf{x}$); $\mathbf{H}$ mappa un punto in un punto ($\mathbf{x}' = \mathbf{H} \mathbf{x}$).

12.3 La matrice fondamentale

Matrice fondamentale

$\mathbf{F} \in \mathbb{R}^{3 \times 3}$ è la generalizzazione di $\mathbf{E}$ per camere **non calibrate** ($\mathbf{K}$ ignota); opera direttamente su pixel:

$$\mathbf{p}'^T \mathbf{F} \mathbf{p} = 0, \quad \mathbf{F} = \mathbf{K}'^{-T} \mathbf{E} \mathbf{K}^{-1}$$

Derivazione algebrica di F da E

Siano $\mathbf{p}, \mathbf{p}'$ le coordinate pixel e $\mathbf{x}, \mathbf{x}'$ le coordinate normalizzate della camera. La matrice intrinseca $\mathbf{K}$ lega i due domini:

$$\mathbf{p} = \mathbf{K} \mathbf{x} \implies \mathbf{x} = \mathbf{K}^{-1} \mathbf{p} \quad \text{e} \quad \mathbf{x}' = \mathbf{K}'^{-1} \mathbf{p}'$$

Sostituendo nell'equazione del vincolo epipolare della matrice essenziale $\mathbf{x}'^T \mathbf{E} \mathbf{x} = 0$:

$$(\mathbf{K}'^{-1} \mathbf{p}')^T \mathbf{E} (\mathbf{K}^{-1} \mathbf{p}) = 0 \implies \mathbf{p}'^T \underbrace{(\mathbf{K}'^{-T} \mathbf{E} \mathbf{K}^{-1})}_{\mathbf{F}} \mathbf{p} = 0$$

Otteniamo la definizione formale della Matrice Fondamentale:

$$\mathbf{F} = \mathbf{K}'^{-T} \mathbf{E} \mathbf{K}^{-1} = \mathbf{K}'^{-T} \mathbf{R}[\mathbf{t}]_{\times} \mathbf{K}^{-1}$$

Proprietà di $\mathbf{F}$

- Opera su coordinate pixel.
- Rango 2, $\det(\mathbf{F}) = 0$.
- 7 DOF.
- Fornisce una “calibrazione debole”.
- $\mathbf{l}' = \mathbf{F}\mathbf{p}$.

$\mathbf{F}$ vs. $\mathbf{E}$ vs. $\mathbf{H}$

$\mathbf{H}$: rango 3, 8 DOF. $\mathbf{F}$: rango 2, 7 DOF. $\mathbf{E}$: rango 2, 5 DOF. Se $\mathbf{K}, \mathbf{K}'$ note: $\mathbf{E} = \mathbf{K}'^T \mathbf{F} \mathbf{K}$.

12.4 Stima della matrice fondamentale

12.4.1 Algoritmo a 8 punti

Ogni corrispondenza dà un'equazione lineare nelle 9 incognite di $\mathbf{F}$. Con ≥ 8 corrispondenze: sistema $\mathbf{A}\mathbf{f} = \mathbf{0} \Rightarrow$ soluzione SVD (autovettore di $\mathbf{A}^T \mathbf{A}$ per il minimo autovalore).

Instabilità numerica

Le colonne di $\mathbf{A}$ hanno magnitudini molto diverse (es. $p_x p'_x$ vs. costante 1) $\Rightarrow$ cattivo condizionamento.

12.4.2 Algoritmo a 8 punti normalizzato

1. **Normalizzare**: traslare i punti al centroide; scalare a distanza media $\sqrt{2}$. Matrici: $\mathbf{T}, \mathbf{T}'$.
2. **Stimare $\hat{\mathbf{F}}$** sui punti normalizzati (algoritmo a 8 punti base).
3. **Imporre rango 2**: SVD di $\hat{\mathbf{F}} = \mathbf{U}\Sigma\mathbf{V}^T$; imporre a zero il minimo valore singolare; ricalcolare $\hat{\mathbf{F}}'$.
4. **Denormalizzare**: $\mathbf{F} = \mathbf{T}'^T \hat{\mathbf{F}}' \mathbf{T}$.

In pratica si combina con **RANSAC** per gestire gli outlier.

12.5 Triangolazione

Dati $\mathbf{P}, \mathbf{P}'$ e corrispondenze $(\mathbf{x}, \mathbf{x}')$:

$$\mathbf{x} \times (\mathbf{P}\mathbf{x}) = \mathbf{0}, \quad \mathbf{x}' \times (\mathbf{P}'\mathbf{x}) = \mathbf{0}$$

Ogni prodotto vettoriale dà 2 equazioni in $\mathbf{x} \in \mathbb{R}^3$. Sistema $4 \times 3 \mathbf{A}\mathbf{x} = \mathbf{0}$ risolto per SVD.

Il problema del rumore

I raggi retroproiettati non si intersecano perfettamente a causa del rumore. Il metodo SVD minimizza un errore algebrico. Il **bundle adjustment** minimizza l'errore di riproiezione (non lineare, più accurato).

12.6 Rettificazione stereo e DSI

La **rettificazione stereo** deforma le due immagini per allineare le linee epipolari orizzontalmente, riducendo il matching da 2D a 1D.

La **Disparity Space Image (DSI)** memorizza i costi di matching $C(i, j)$ per ogni coppia di pixel corrispondenti nella stessa scanline. La **programmazione dinamica** trova il percorso ottimale:

$$C(i, j) = \min \begin{cases} C(i-1, j-1) + D(i, j) & \text{(match)} \\ C(i-1, j) + c_{\text{occ}} & \text{(occlusione da destra)} \\ C(i, j-1) + c_{\text{occ}} & \text{(occlusione da sinistra)} \end{cases}$$

Trattando le scanline indipendentemente si possono generare artefatti di striatura.

12.7 Da F a moto della camera

Se $\mathbf{K}, \mathbf{K}'$ sono note, si recupera $\mathbf{E} = \mathbf{K}'^T \mathbf{F} \mathbf{K}$ e si decompone: 4 soluzioni possibili per $(\mathbf{R}, \mathbf{t})$. La corretta si determina verificando che i punti 3D siano davanti a entrambe le camere.

Per 3 viste: **tensore trifocale**; per 4 viste: **tensore quadrifocale**.

Chapter 13

Vision Transformers e Architetture Avanzate

13.1 Dal NLP alla Visione: Motivazione

I **Transformers** sono stati introdotti originariamente per il Natural Language Processing (NLP) nel paper "*Attention Is All You Need*" (Vaswani et al., 2017). La loro estensione alla visione artificiale deriva dalla capacità del meccanismo di **Self-Attention** di modellare le interazioni tra coppie di elementi in una sequenza in un'unica operazione globale, eliminando il bias di località tipico delle CNN.

13.2 Meccanismo di Self-Attention

Dato un insieme di vettori di input $X \in \mathbb{R}^{n \times d_{model}}$, il modello impara tre proiezioni lineari tramite matrici di peso $W_Q, W_K \in \mathbb{R}^{d_{model} \times d_k}$ e $W_V \in \mathbb{R}^{d_{model} \times d_v}$:

$$Q = XW_Q \quad (\text{Queries}), \quad K = XW_K \quad (\text{Keys}), \quad V = XW_V \quad (\text{Values})$$

Scaled Dot-Product Attention

La risposta di attenzione viene calcolata moltiplicando la matrice delle Queries con la trasposta delle Keys, applicando un fattore di scala e la funzione softmax sui Values:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Perché il fattore di scala $\frac{1}{\sqrt{d_k}}$?

Per valori elevati della dimensione d_k , il prodotto scalare QK^T tende ad assumere valori numericamente grandi. Questo spinge la funzione softmax in regioni a gradiente estremamente piccolo (saturazione), provocando il fenomeno del *vanishing gradient* durante l'addestramento. La divisione per $\sqrt{d_k}$ stabilizza i gradienti mantenendo la varianza dei prodotti scalari vicina a 1.

13.2.1 Multi-Head Attention (MHA)

Invece di calcolare un'unica funzione di attenzione, la **Multi-Head Attention** esegue h meccanismi di attenzione in parallelo. Ogni "testa" (head) proietta Q, K, V in sottospazi di rappresentazione differenti, consentendo al modello di focalizzarsi contemporaneamente su relazioni spaziali o semantiche diverse:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

dove $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ e $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

13.2.2 Positional Encoding

La Self-Attention è **permutation-equivariant**: è del tutto indifferente all'ordine d'ingresso dei token. Poiché nella visione artificiale la posizione geometrica dei pixel è fondamentale, occorre iniettare l'informazione spaziale sommando agli embedding di input un vettore di **Positional Encoding** (funzioni sinusoidali fisse o embedding posizionali 1D/2D apprendibili).

13.2.3 Struttura del Transformer Encoder Block

Ogni blocco dell'encoder applica in sequenza:

1. Multi-Head Self-Attention con connessione residuale e Layer Normalization.
2. Feed-Forward Network (FFN) composta da due strati lineari con non-linearità (es. GELU o ReLU), integrata da connessione residuale e Layer Normalization.

13.3 Vision Transformer (ViT)

Il **Vision Transformer** (Dosovitskiy et al., 2021) applica l'architettura del Transformer Encoder direttamente a sequenze di patch d'immagine.

Pipeline di ViT

1. **Patch Extraction**: un'immagine $H \times W \times C$ viene divisa in $N = \frac{HW}{P^2}$ patch non sovrapposti di dimensione $P \times P$ (es. 16×16).
2. **Linear Embedding**: ogni patch viene appiattito in un vettore P^2C e proiettato linearmente in una dimensione d_{model} .
3. **Class Token ([CLS])**: un token apprendibile $[CLS]$ viene aggiunto in testa alla sequenza; la sua rappresentazione finale all'uscita dell'encoder verrà usata per la classificazione globale.
4. **Positional Encoding**: si sommano vettori posizionali 1D apprendibili.
5. **Transformer Encoder**: la sequenza passa attraverso L blocchi residuali MHA + FFN.
6. **Classification Head**: un perceptron multistrato (MLP) viene applicato all'output del token $[CLS]$.

CNN vs. ViT: Inductive Biases

- **CNN**: possiedono forti *inductive biases* specifici per le immagini, ovvero la **Località** (pixel vicini sono correlati) e l'**Equivarianza alla Traslazione** (stessi filtri applicati ovunque). Imparano molto velocemente anche su piccoli dataset.
- **ViT**: hanno *inductive biases* visivi molto **deboli**. Non assumono innatamente la geometria 2D dell'immagine.
- **Conseguenza**: i ViT richiedono pre-addestramento su dataset di scala massiva (es. ImageNet-21k o JFT-300M) per apprendere da soli le relazioni spaziali. Su grande scala superano nettamente le CNN modellando interazioni globali $O(N^2)$.

13.4 Swin Transformer: Shifted-Window Attention

Il ViT classico calcola l'attenzione globale tra tutti i patch, con complessità quadratica $O(N^2)$ rispetto alla risoluzione. Lo **Swin Transformer** (Liu et al., 2021) introduce l'attenzione a finestre traslate per ottenere complessità lineare $O(N)$ e mappe di feature gerarchiche multi-scala.

Meccanismi di Swin Transformer

- **Local Windows**: l'immagine viene divisa in finestre non sovrapposte $M \times M$ (es. 7×7 patch). La Self-Attention viene calcolata esclusivamente all'interno di ciascuna finestra $\Rightarrow$ Complessità $O(N)$.
- **Shifted Window Attention**: tra strati consecutivi, le finestre vengono traslate di $(\lfloor M/2 \rfloor, \lfloor M/2 \rfloor)$ pixel. Questo consente connessioni *cross-window* tra patch adiacenti appartenenti a finestre diverse, mantenendo la complessità computazionale lineare.
- **Hierarchical Feature Maps**: accorpa patch adiacenti nei layer profondi costruendo una piramide di feature simile alle CNN, ideale per detection e segmentazione.

13.5 DETR: End-to-End Object Detection with Transformers

DETR (Carion et al., 2020) riformula l'Object Detection come un problema di predizione diretta d'insieme (*set-prediction*), eliminando componenti euristiche manuali come le *anchor boxes* e la *Non-Maximum Suppression (NMS)*.

Architettura di DETR

1. **CNN Backbone**: estrae una mappa di caratteristiche dall'immagine (es. $H/32 \times W/32 \times C$).
2. **Transformer Encoder**: elabora le feature spaziali appiattite insieme a positional encodings.
3. **Transformer Decoder**: riceve N_q **Object Queries** apprendibili (vettori posizionali trasformati che "interrogano" la scena) ed esegue Cross-Attention

con l'output dell'encoder.

4. **Prediction Heads:** MLP lineari mappano ciascuna query in una bounding box (x, y, w, h) e una classe (incluso la classe "no object").
5. **Bipartite Matching Loss:** usa l'**algoritmo Ungherese** durante l'addestramento per assegnare in modo unico ogni predizione a un oggetto ground-truth.

Chapter 14

Modelli Generativi Profondi: GANs e Diffusione

14.1 Formulazione generale

I modelli generativi profondi apprendono la distribuzione di probabilità sottostante $p_{\text{data}}(x)$ a partire da dati ad alta dimensione (immagini) per campionare nuove istanze sintetiche $x \sim p_{\theta}(x)$.

14.2 Generative Adversarial Networks (GANs)

Un'architettura **GAN** (Goodfellow et al., 2014) è composta da due reti neurali addestrate in un gioco minimax avversario a due giocatori:

- **Generatore (G)**: mappa un vettore latente stocastico $z \sim p_z$ in un campione sintetico $G(z)$.
- **Discriminatore (D)**: calcola la probabilità $D(x) \in [0, 1]$ che un campione provenga dai dati reali p_{data} piuttosto che da G .

Funzione Obiettivo Minimax delle GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

All'equilibrio di Nash, G ricostruisce esattamente la distribuzione p_{data} e $D(x) \equiv 1/2$.

Problematiche di Addestramento e Fallimenti

- **Mode Collapse**: G impara a produrre solo una piccola sotto-varietà di immagini reali che ingannano D , ignorando la diversità del dataset.
- **Vanishing Gradients**: se D diventa troppo forte rispetto a G , i gradienti per G crollano a zero.
- **Wasserstein GAN (WGAN)**: risolve le instabilità sostituendo la divergenza JS con la *Distanza di Wasserstein-1* (Earth Mover's Distance) e imponendo il vincolo di 1-Lipschitzianità a D .

14.3 Denoising Diffusion Probabilistic Models (DDPM)

I modelli di diffusione (Ho et al., 2020) definiscono un processo di generazione basato su due catene stocastiche di Markov.

14.3.1 Forward Process (Noising)

Un processo fisso (non apprendibile) che corrompe progressivamente un'immagine reale x_0 aggiungendo rumore Gaussiano isotropo in T passi temporali:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

dove $\{\beta_t \in (0, 1)\}_{t=1}^T$ è una schedula di varianza prefissata.

Reparameterization Trick nei DDPM

Definendo $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, è possibile campionare direttamente l'immagine rumorosa x_t al generico passo t a partire da x_0 in forma chiusa senza iterare tutti i passi precedenti:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

14.3.2 Reverse Process (Denoising)

Il processo inverso è una catena di Markov apprendibile parametrizzata da una rete neurale $\epsilon_\theta(x_t, t)$ (solitamente un'architettura **U-Net**):

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Loss Semplificata di DDPM

Invece di prevedere l'immagine pulita, la rete viene addestrata a **prevedere il rumore ϵ^{**} aggiunto al passo t :

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

- **Diffusione:** addestramento stabile (nessun gioco avversario), evita il mode collapse e garantisce un'altissima diversità dei campioni.
- **Svantaggio:** campionamento iterativo lento ($T \approx 1000$ passaggi U-Net all'inferenza). Mitigato da metodi come **DDIM** (deterministic sampling a ~ 50 passi) e modelli di consistenza.

!

Chapter 15

Deep Learning per Video e Dati Temporali

15.1 Il Video come Segnale Spazio-Temporale

Un video è un segnale 3D rappresentato da un tensore di dimensione $T \times H \times W \times C$, dove T è la dimensione temporale (sequenza di frame) e $H \times W$ sono le coordinate spaziali.

15.2 Architetture per Video Recognition

- **Single-Frame Models:** applicano una CNN 2D indipendentemente su ogni frame e aggregano le predizioni tramite pooling (ignore l'ordine temporale).
- **2D CNN + RNN / LSTM:** estraggono feature da ciascun frame con una CNN 2D e passano la sequenza a una rete ricorsiva per modellare le dipendenze temporali.
- **3D CNNs (es. C3D):** estendono la convoluzione 2D aggiungendo una dimensione temporale ai kernel ($T_k \times K \times K$), elaborando congiuntamente spazio e tempo.
- **Two-Stream Networks:** dividono l'analisi in due rami separati:
 1. ***Spatial Stream*:** elabora i frame RGB (apparenza).
 2. ***Temporal Stream*:** elabora stack di campi di **Optical Flow** (movimento).

15.3 TimeSformer: Video Vision Transformer

Estende il Vision Transformer al video trattando regioni spazio-temporali 3D come *tubelets*. Per evitare l'esplosione della memoria dovuta all'attenzione congiunta spazio-tempo $O((T \times N)^2)$, introduce la **Divided Space-Time Attention**:

1

Chapter 16

Vision-Language Models (VLMs)

16.1 Motivazione e Strategie di Fusione

I modelli unimodali estraggono feature spaziali ma mancano di ragionamento semantico linguistico. I **Vision-Language Models (VLMs)** rappresentano congiuntamente immagini e testo per compiti quali *Visual Question Answering (VQA)*, *Image Captioning* e *Zero-Shot Recognition*.

Strategie di Fusione Multimodale

- **Early Fusion:** concatenazione dei token visivi e testuali al primo strato di un unico Transformer condiviso.
- **Intermediate Fusion:** encoder dedicati separati le cui feature intermedie vengono fuse a metà rete tramite moduli di Cross-Attention (es. Flamingo).
- **Late Fusion:** elaborazione del tutto indipendente fino agli embedding finali, confrontati nello spazio latente (es. CLIP).

Modality Gap

Il **Modality Gap** indica la divergenza geometrica tra lo spazio delle feature visive e quello delle feature testuali. Un VLM impara una proiezione condivisa in cui coppie immagine-testo corrispondenti si trovino vicine nello spazio latente.

16.2 CLIP: Contrastive Language-Image Pre-training

CLIP (Radford et al., 2021) è un VLM a Late Fusion addestrato su 400 milioni di coppie immagine-testo pubblicamente disponibili sul web tramite **Apprendimento Contrastivo**.

InfoNCE Loss in CLIP

Dato un batch di N coppie (Immagine I_k , Testo T_k), si calcola la matrice di similarità del coseno $N \times N$. La loss massimizza i valori lungo la diagonale (coppie

positive) e minimizza gli elementi fuori diagonale:

$$\mathcal{L}_{I \rightarrow T} = - \sum_{k=1}^N \log \frac{\exp(I_k \cdot T_k / \tau)}{\sum_{j=1}^N \exp(I_k \cdot T_j / \tau)}, \quad \mathcal{L}_{\text{CLIP}} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I})$$

dove τ è un parametro di temperatura apprendibile.

16.2.1 Zero-Shot Transfer

Durante l'inferenza, non occorre alcun addestramento specifico. I nomi delle classi vengono trasformati in prompt testuali tramite *prompt engineering* (es. *"a photo of a {class}"*). L'immagine di input viene classificata assegnandole la classe del prompt testuale il cui embedding presenta la **massima similarità del coseno** con l'embedding dell'immagine.

Chapter 17

Medical Imaging e Segmentazione Avanzata

17.1 Il Medical Data Gap

A differenza dei dataset visivi naturali (ImageNet), l'imaging medico presenta forti limitazioni: volumetria ridotta, annotazioni estremamente costose poiché richiedono medici specialisti, vincoli normativi rigidi sulla privacy (GDPR/HIPAA) e un forte **Domain Shift** dovuto a scanner e protocolli di acquisizione differenti tra ospedali.

17.2 Architetture di Segmentazione Biomedica

U-Net

La **U-Net** (Ronneberger et al., 2015) è un'architettura Encoder-Decoder simmetrica:

- **Encoder** (contrazione): estrae feature semantiche ad alto livello tramite convoluzioni e pooling.
- **Decoder** (espansione): ripristina la risoluzione spaziale originale tramite upsampling.
- **Skip Connections**: concatenano le mappe di caratteristiche ad alta risoluzione dell'encoder direttamente con i corrispondenti livelli del decoder, preservando i dettagli spaziali dei bordi delle lesioni.

17.3 Funzioni di Loss per Dati Sbilanciati

Nel settore medico, lo sfondo occupa la maggior parte dell'immagine, mentre il tumore/lesione rappresenta solo una frazione minuscola di pixel.

Dice Loss e Focal Loss

- **Dice Loss**: ottimizza direttamente l'overlap geometrico tra la maschera prevista y e la ground-truth t , risultando insensibile allo sbilanciamento delle

classi:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{C} \sum_{c=0}^{C-1} \frac{2 \sum_n t_n^c y_n^c}{\sum_n (t_n^c + y_n^c)}$$

- **Focal Loss**: abbassa il peso dei pixel facili (sfondo) focalizzando la back-propagation sugli esempi difficili (i confini della lesione):

$$\mathcal{L}_{\text{Focal}} = - \sum_{n=1}^N (1 - p_{t,n})^\gamma \log(p_{t,n}), \quad (\text{tipicamente } \gamma = 2)$$

17.4 Metriche di Valutazione

* **Dice Score (DSC)**: equivalent all'F1-score $\implies \text{DSC} = \frac{2|X \cap Y|}{|X| + |Y|}$. * **Hausdorff Distance (HD)**: misura la deviazione peggiore superficie-superficie tra i contorni previsti e quelli reali:

$$\text{HD}(X, Y) = \max \left\{ \max_{x \in X} \min_{y \in Y} \|x - y\|, \max_{y \in Y} \min_{x \in X} \|x - y\| \right\}$$

Chapter 18

Monocular Depth Estimation (MDE)

18.1 Definizione e Natura Ill-Posed

La **Monocular Depth Estimation (MDE)** consiste nel prevedere una mappa di profondità densa $Z(x, y)$ a partire da un'unica immagine RGB 2D.

Perché MDE è un problema mal posto?

La proiezione da 3D a 2D perde un grado di libertà (la profondità lungo il raggio ottico). Infiniti mondi 3D differenti possono proiettarsi nella stessa identica immagine 2D. Il problema è quindi matematicamente mal posto e richiede che il modello apprenda *priors* visivi dal contesto.

18.2 Indizi Monoculari (Monocular Cues)

* **Prospettiva lineare**: linee parallele che convergono verso il punto di fuga. * **Dimensione relativa**: oggetti noti appaiono più piccoli se lontani. * **Gradiente di texture**: la grana delle superfici diventa più densa a distanza. * **Occlusione**: gli oggetti in primo piano nascondono quelli sullo sfondo.

18.3 Depth Anything v2

Modello state-of-the-art basato su una pipeline **Teacher-Student** di pseudo-labelling:

1. Si addestra un modello *Teacher* su dati sintetici (profondità perfetta senza rumore).
2. Il Teacher genera pseudo-labels dense su un dataset immenso di immagini reali non etichettate.
3. Si addestra un modello *Student* su queste pseudo-labels, garantendo forte generalizzazione.

Chapter 19

Efficienza nel Deep Learning

19.1 Training Efficiente vs. Inferenza Efficiente

* **Efficient Training**: riduce il tempo di addestramento su GPU/TPU (es. *mixed-precision FP16*, *gradient checkpointing*). * **Efficient Inference**: minimizza latenza e memoria durante la distribuzione su dispositivi mobile o embedded.

19.2 Tecniche di Inferenza Efficiente

Tecniche di Compressione

1. **Architetture Compatte**: uso di *depthwise separable convolutions* (es. MobileNet) per separare il filtraggio spaziale dalla combinazione dei canali.
2. **Pruning**: rimozione dei pesi meno importanti.
 - *Structured Pruning*: rimuove interi canali/filtri $\Rightarrow$ velocizzazione immediata su hardware standard.
 - *Unstructured Pruning*: azzerà pesi singoli $\Rightarrow$ richiede hardware specializzato per matrici sparse.
3. **Knowledge Distillation**: addestramento di una rete piccola (*Student*) affinché imiti la distribuzione delle probabilità umane o le feature intermedie di una rete grande (*Teacher*).
4. **Quantizzazione**: riduzione della precisione numerica dei pesi e delle attivazioni. Passare da **FP32** ad **INT8** riduce l'impronta di memoria di $4\times$ e abilita l'aritmetica intera sui processori.

19.3 Metriche e Trade-off

* **MACs**: operazioni di *Multiply-Accumulate*. * **FLOPs**: operazioni in virgola mobile ($\approx 2 \times$ MACs). * **Pareto Frontier**: la curva che rappresenta le configurazioni di modelli ottimali per cui non esiste un'architettura contemporaneamente più precisa e più veloce.